



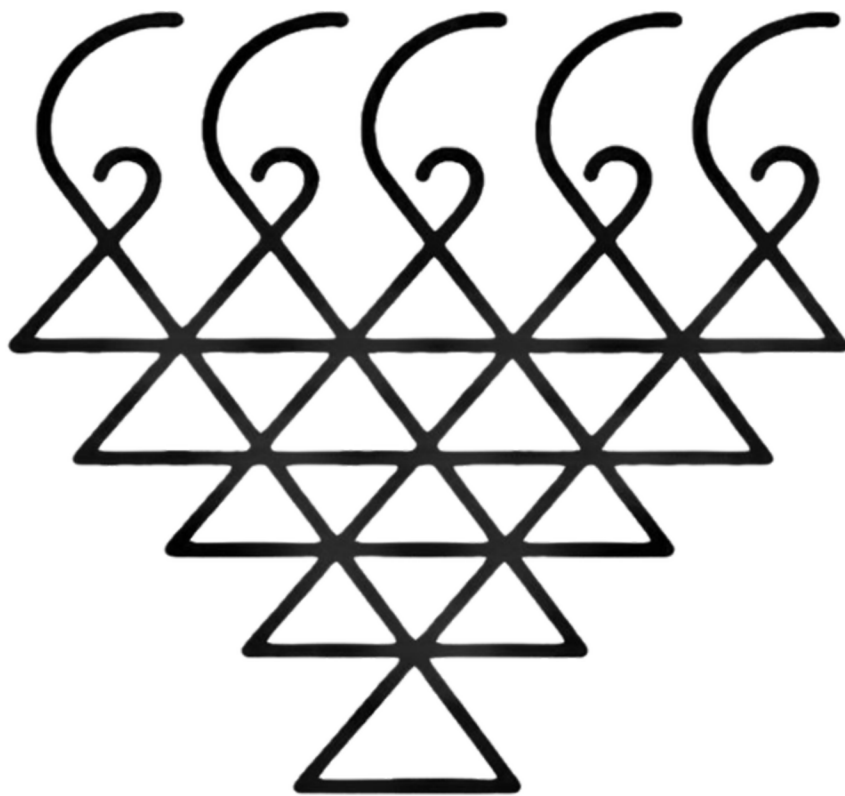
FirstBitSolutions.com

... Learn IT, Bit by Bit ...

Training | Placement | Internship

ज्ञानेश्वरी

MySQL



**“Every Student's Story,
Must be a Success Story!”**

1. Basics of MySQL.....	5-6
• What is Database.....	5
• What is DBMS.....	5
• What is RDBMS.....	5
• What is MySQL.....	5
• What is SQL	5
• Analogy for DB, DBMS, RDBMS, MySQL and SQL	5
• Query execution flow	6
2. Keys in MySQL.....	7
• Candidate Key	7
• Primary Key	7
• Alternate Key	7
• Composite Key	7
• Foreign Key	7
3. Cardinality in MySQL.....	8-9
• One to One Cardinality	8
• One to Many Cardinality	8
• Many to Many Cardinality	8
• Establishing connection between	8
4. Constraints in MySQL	10-14
• PRIMARY KEY Constraint.....	10
• FOREIGN KEY Constraint.....	11
• UNIQUE Constraint	12
• DEFAULT Constraint.....	13
• CHECK Constraint	13
• NOT NULL Constraint	14
5. Normalisation in MySQL	15-20
• Why Normalisation.....	15
• 1 st Normal Form.....	16
• 2 nd Normal Form.....	16
• 3 rd Normal Form	17
• Boyce-Codd Normal Form(BCNF)	18
6. Data Definition Language(DDL)	21-25
• CREATE Command.....	21
• Data Types in MySQL	21
• ALTER Command.....	22
• DROP Command	23
• TRUNCATE Command	24
• RENAME Command.....	24
7. Data Manipulation Language	26-28
• INSERT Command.....	26
• WHERE Clause.....	26
• UPDATE Command	27
• Delete Command	27
• Difference between DROP, DELETE and TRUNCATE	28

8. Data Query Language.....	28-29
• SELECT.....	28
• Variation of Select.....	28
• WHERE clause with Operators.....	29
9. Pattern Matching(LIKE Operator).....	29-30
• Definition	29
• Wildcard used with LIKE	29
10. DISTINCT Keyword	30-31
• Why we use DISTINCT.....	30
• What is DISTINCT	30
• Syntax & example	30
11. ORDER BY Clause	32-33
• Definition	32
• Syntax & example	32
• LIMIT Clause.....	32
• OFFSET Keyword	33
• Pagination Concept.....	33
12. Aggregation Function	33-34
• Why we use Aggregation Function.....	33
• Definition	34
• Example for each function	34
• Key points	34
13. GROUP BY Clause.....	35-37
• Why we use GROUP BY Clause.....	35
• Definition	35
• How it works.....	35
• Analogy	35
• Using multiple columns in GROUP BY	36
• Combining GROUP BY with ORDER BY	36
• Filtering groups using HAVING Clause.....	36
14. Built-in Scaler Functions.....	38-42
• String Functions	38
• Numeric Functions.....	39
• Date & Time Function	41
15. Subquery.....	43-45
• What is Subquery.....	43
• Syntax & Example	43
• Type of Subquery	44
i. Single-row Subquery	44
ii. Multiple-row Subquery	44
iii. Correlated Subquery	45
16. Joins.....	46-55
• CROSS JOIN.....	46
• INNER JOIN	47
• LEFT OUTER JOIN	48
• RIGHT OUTER JOIN	49
• Self-Join	50
• Full Outer Join	51

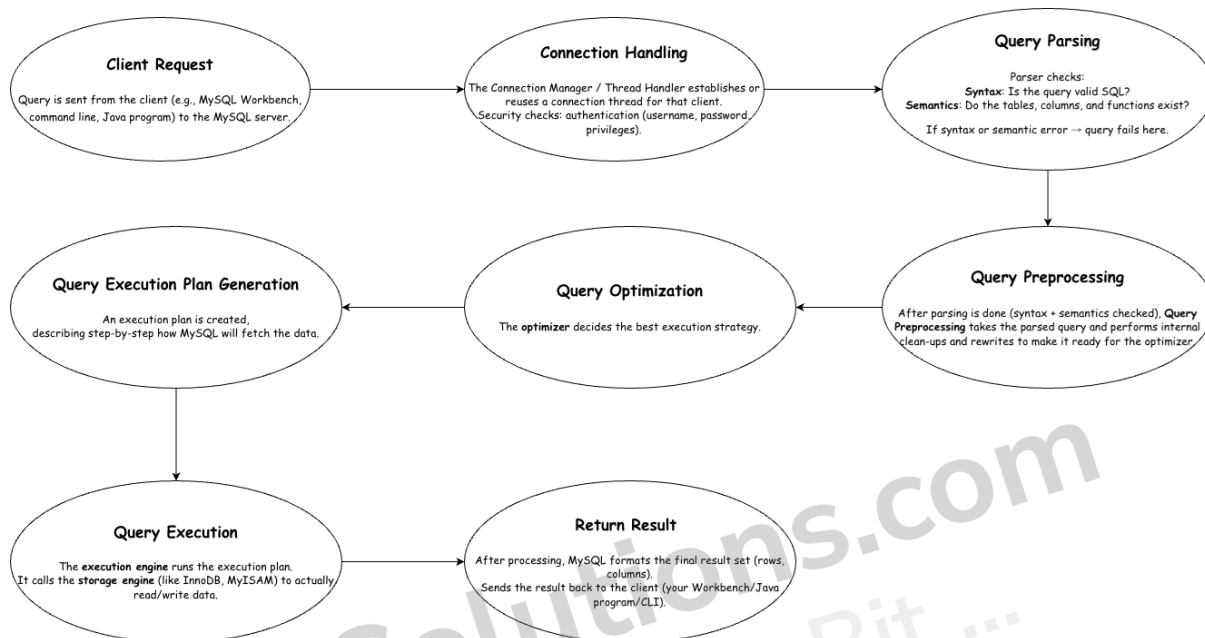
• Summary of all joins.....	53
• UNION	53
• UNION ALL	53
• Difference between UNION & UNION ALL.....	55
17. ACID Properties.....	55-56
• Atomicity.....	55
• Consistency	55
• Isolation.....	56
• Durability.....	56
18. Transaction Control Language.....	56-58
• COMMIT Command.....	56
• ROLLBACK Command.....	56
• SAVEPOINT Command	57
• SET AUTOCOMMIT Statement	57
19. Data Control Language	58-60
• CREATE USER Command	59
• GRANT Command	59
i. Granting all privilege.....	59
ii. Common privilege.....	59
• REVOKE Command	60
20. VIEW	60-63
• What is VIEW	60
• Why we use VIEW	60
• Syntax & Example	60
• Modifying a VIEW.....	61
• Dropping a VIEW	61
• Types of VIEW	61
• Updatable and Non-Updatable View	62
21. INDEX.....	63-64
• Definition	63
• Analogy	63
• Syntax.....	63
• Type of Index.....	63
22. PL/SQL	65
• What is PL/SQL	65
• Analogy	65
23. Stored Procedure	65-68
• What is Stored Procedure	65
• Why to change DELIMITER.....	65
• Managing Stored Procedure	66
i. View Existing procedure	66
ii. To see the code of specific procedure	66
iii. Delete (DROP) a procedure	66
iv. Safe deletion using IF EXIST.....	66
• Stored Procedure Parameter	67
i. IN Parameter	67
ii. OUT Parameter.....	67
iii. INOUT Parameter	67

• Summary of Stored Procedure	68
24. Control Flow Statement	69-71
• IF-ELSEIF-ELSE	69
• CASE Statement.....	69
• LOOP	70
• WHILE.....	70
• REPEAT	71
• Summary.....	71
25. Stored Function	72-74
• Definition	72
• Syntax & Example	72
• Managing Stored Function.....	73
i. View existing function.....	73
ii. To view the code of particular function	73
iii. Delete (Drop) function	73
iv. Safe deletion using IF EXIST.....	73
• Summary.....	73
• Difference between function and procedure.....	73
26. Trigger	74-77
• Definition	74
• Syntax & Examples.....	74
• Understanding of OLD and NEW.....	75
• Managing Trigger	75
i. View existing trigger	75
ii. View trigger definition.....	75
iii. Delete (DROP) a Trigger.....	75
iv. Safe deletion using IF EXIST.....	76
• Summary.....	76
• Types of Trigger.....	76
• Notes on Trigger.....	76
27. Cursor	77-79
• What is Cursor	77
• Why we need Cursor	77
• Use Case	78
• Examples.....	78
• Summary.....	79



- What is Database?
 - A Database is a collection of related data organized in a way that data can be easily Accessed, Managed and Updated. Any piece of information can be a data.
 - For example name of your school. Database is actually a place where related piece of information is stored and various operations can be performed on it.
- What is DBMS?
 - A DBMS is a software system that allows creation, definition and manipulation of database. DBMS is actually a tool used to perform any kind of operation on data in database. DBMS also provides protection and security to database.
 - It maintains data consistency in case of multiple users. Here are some
 - examples of popular DBMS, MySQL, Oracle, Sybase, Microsoft Access and IBM DB2 etc
- What is RDBMS?
 - A Relational Database management System(RDBMS) is a database management system based on relational model introduced by E.F Codd.
 - In relational model, data is represented in terms of tables.
 - RDBMS is used to manage relational databases. Relational database is a collection of organized set of tables from which data can be accessed easily.
 - Relational Database is most commonly used database. It consists of number of tables and each table has its own primary key.
 - Different Objects of RDBMS Structure :
 - Database:
 - It is a collection of related data organized for easy access, management, and updating.
 - It contains multiple tables.
 - Table: An entity name decides the name of the table.
 - Example : "Employee" entity becomes the Employee table.
 - Field: Entities have many attributes and that attributes will decide the name of columns /fields of your table.
 - Record: One row will define record for one entity/tables.
- What is MySQL?
 - MySQL is an open-source Relational Database Management System (RDBMS) that uses Structured Query Language (SQL) to store, manage, and retrieve data in the form of tables.
 - It follows a client-server architecture, supports multiple platforms, and is widely used in web applications and enterprise software for reliable and efficient data management.
- What is SQL?
 - SQL is the language used to communicate with and manipulate data in a database. It is a query language, similar to English, used to manage data.
- Analogy for DB, DBMS, MySQL and SQL
 - Database(DB) → Think of it as a library where all the books (data) are stored.
 - DBMS (Database Management System) → The librarian who organizes, manages, and helps you retrieve the right book efficiently.
 - MySQL → A specific librarian brand/system (e.g., "City Library") that uses DBMS rules. It's one implementation of DBMS.
 - SQL (Structured Query Language) → The language you use to talk to the librarian. For example, you ask:

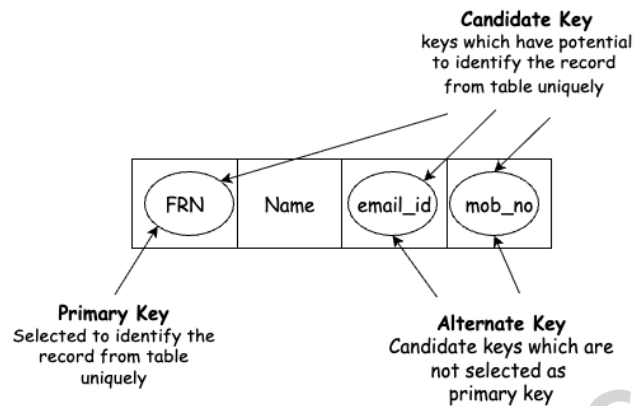
- “Give me all books written by author J. K Rowling” → SELECT *
FROM books WHERE author = 'J. K. Rowling';
- Query execution flow



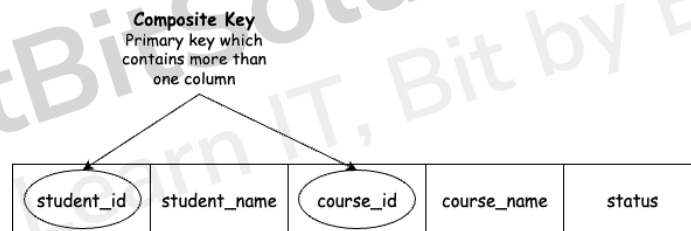
FirstBitSolutions.com
... Learn IT, Bit by Bit ...

Keys in MySQL

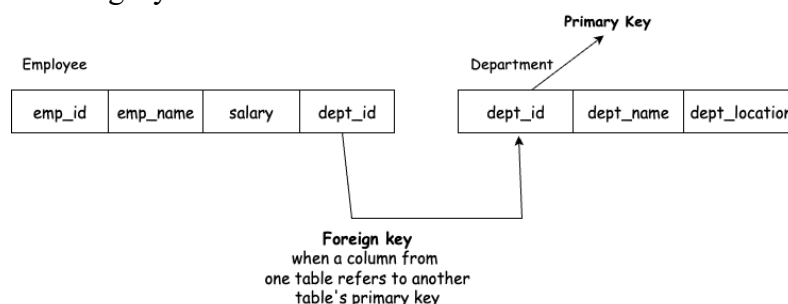
- **Candidate Key:** A column or attribute or field which has potential to identify the records from your table uniquely, but can have duplicate and null values.
- **Primary Key:** Primary Key is a candidate key which is selected to identify the record from the table uniquely and doesn't contain any duplicate and null values.
- **Alternate Key:** The key which are not selected for being a primary key but have potential to be selected as a primary key but have possibility to have null or same data.



- **Composite Key:** A primary key which contains more than one column which is called composite key. In your table when one column / attribute / field is not enough to identify a record uniquely, that time we have to make a primary key which contains more than one column.



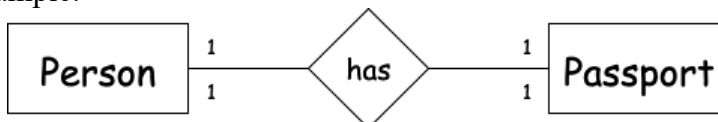
- **Foreign Key:** When a column from table references to the another's table primary key, It is used to establish and enforce a relationship between the two tables, ensuring referential integrity of the data.



Cardinality in Relationship in MySQL

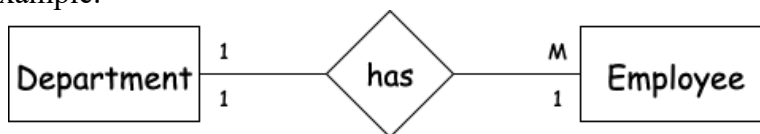
- One to One

- If in your database two tables/entities have one-to-one relation from entity A to entity B and one-to-one relation from entity B to entity A, the ultimate relation between these two entities is _____.
- Example:



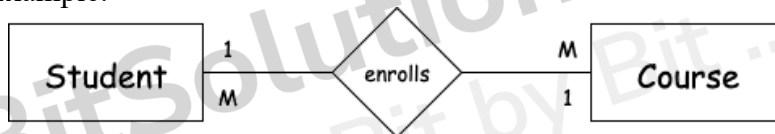
- One to Many

- In your database if two entities/tables have one-to-many relation from A entity to B entity, and one-to-one from B entity to A entity, then the ultimate relation is _____.
- Example:



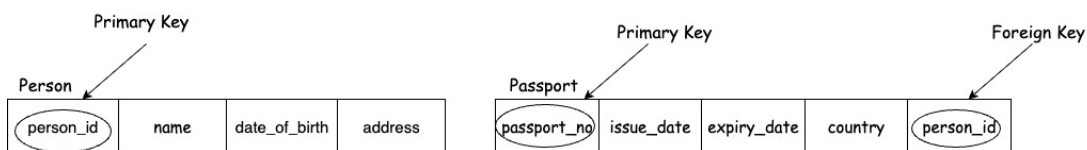
- Many to Many

- In your database if two entities/tables have one-to-many relation from A entity to B one time and then ultimate relationship is _____.
- Example:



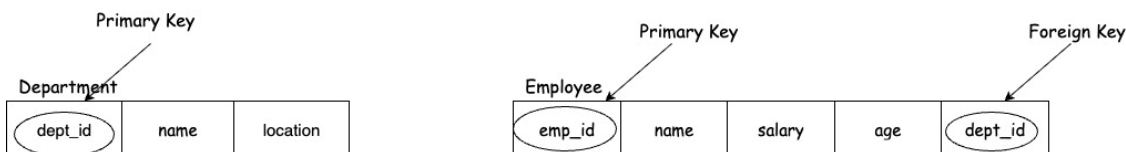
- Establishing the connection between tables:

- Now we have defined the cardinality in relationship between entities, now it is time to establish the relationship that we will do using _____.
- If your two entities have one-to-one relation, then the foreign key goes into the table which is fully dependent on another table.



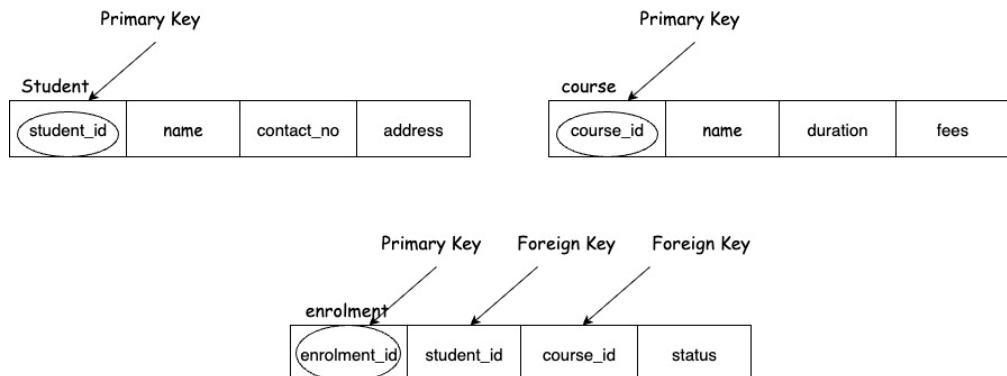
(Foreign key goes in passport table because it is fully dependent on person.)

- In a one-to-many relationship, the foreign key goes in the table that is on the '_____' side.



(Foreign key goes in employee table because it is on 'many' side)

- If two entities have a many-to-many relationship, the foreign key is not placed in either table directly. Instead, we create a third table, called a _____ (or junction) table, which contains the foreign keys from both tables.



(Foreign keys which refers both tables(student & course) goes into bridge or junction table(enrolment))

FirstBitSolutions.com
... Learn IT, Bit by Bit ...

Constraints in MySQL

- Why Constraint?
 - The constraint in MySQL is used to specify the rules that allows or restricts what values/data will be stored in the table.
 - They provide a suitable method to ensure data accuracy and integrity inside the table.
 - It also helps to limit the type of data that will be inserted inside the table.
 - If any interruption occurs between the constraint and data action, the action is terminated.
 - Types of constraints
 - Primary Key
 - Foreign Key
 - Not Null
 - Check
 - Unique
 - Default
- How to apply constraints

```
CREATE TABLE newTableName(
  columnName dataType constraint,
  columnName dataType constraint,
  columnName dataType constraint,
  .....
);
```

- Primary Key
 - The primary key follows these rules:
 - A primary key must contain unique values. If the primary key consists of multiple columns(composite key), the combination of values in these columns must be unique.
 - A primary key column cannot have NULL values. Any attempt to insert or update NULL to primary key columns will result in an error.
 - Note that MySQL implicitly adds a NOT NULL constraint to primary key columns.
 - A table can have one and only one primary key.
 - How to apply primary key
 - Column level(Inline)

```
CREATE TABLE newTableName(
  primary_key_column_name dataType PRIMARY KEY,
  .....
);
```

```
CREATE TABLE Students (
  studentID INT PRIMARY KEY,
  .....
);
```

- Add primary key to an existing table

```
ALTER TABLE table_name
ADD PRIMARY KEY (column_name);
```

```
ALTER TABLE Students
ADD PRIMARY KEY (studentID);
```

- Table Level (after columns)

```
CREATE TABLE newTableName(
    primary_key_column1_name dataType,
    primary_key_column2_name dataType,
    .....
    PRIMARY KEY(column_list)
);
```

```
CREATE TABLE Enrollment (
    studentID INT,
    courseID INT,
    enrollmentDate DATE,
    grade CHAR(2),
    PRIMARY KEY (studentID, courseID)
);
```

- How to drop primary key

```
ALTER TABLE Students
DROP PRIMARY KEY;
```

- Foreign Key
 - A foreign key is called a constraint because it puts a rule on a column:
 - You can only enter values that already exist in another table's primary key.
 - This rule keeps the link between the two tables correct.
 - Example:
 - In the Employee table, dept_id is a foreign key.
 - It must match with a real dept_id in the Department table.
 - You can't add an employee with a dept_id that doesn't exist.
 - Foreign keys can control what happens if the main table's row is deleted or updated:
 - ON UPDATE in Foreign key
 - When a Foreign Key is defined, you can specify what should happen to the rows in the referencing table when the key value in the main table is updated.
 - Options for ON UPDATE:



1. CASCADE → Updates the foreign key value in the referencing table to match the new value in the main table.
2. SET NULL → Sets the foreign key value to **NULL** in the referencing table.
3. RESTRICT/ NO ACTION → It **prevents** the update in the main table if related rows exist in the referencing table.

```
CREATE TABLE Department (
    deptID INT PRIMARY KEY,
    deptName VARCHAR(50)
);
```

```
CREATE TABLE Employee (
    empID INT PRIMARY KEY,
    empName VARCHAR(50),
    deptID INT,
    FOREIGN KEY (deptID) REFERENCES Department(deptID)
    ON UPDATE CASCADE/SET NULL/RESTRICT/NO ACTION
);
```

- ON DELETE in Foreign key

- When a Foreign Key is defined, you can specify what should happen to the rows in the referencing table when a row in the main table is deleted.
- Options for ON DELETE:

1. CASCADE → Delete the referencing table rows automatically.
2. SET NULL → Set the foreign key in referencing rows to NULL.
3. RESTRICT / NO ACTION → Prevent deletion in the main table if referencing rows exist.

```
CREATE TABLE Department (
    deptID INT PRIMARY KEY,
    deptName VARCHAR(50)
);
```

```
CREATE TABLE Employee (
    empID INT PRIMARY KEY,
    empName VARCHAR(50),
    deptID INT,
    FOREIGN KEY (deptID) REFERENCES Department(deptID)
    ON DELETE CASCADE/SET NULL/RESTRICT/NO ACTION
);
```

- So, foreign key = rule (constraint) that connects two tables correctly.

- Unique Constraint

- If you make any column unique, then the value we are inserting in that column must be unique (no duplicate).
- Applied directly when defining a column.

```
CREATE TABLE Student (
    email VARCHAR(100) UNIQUE
);
```



- Defined separately, often used when you want to make multiple columns together unique.

```
CREATE TABLE Student (
    id INT,
    email VARCHAR(100),
    CONSTRAINT uq_email UNIQUE (email)
);
```

- Add a unique constraint to an existing table.

```
ALTER TABLE Student
ADD CONSTRAINT uq_email UNIQUE (email);
```

- How to drop it.

```
ALTER TABLE Student
DROP INDEX uq_email;
```

- Default Constraint

- If we apply a DEFAULT constraint to a column, then when a user does not provide a value, the column will automatically take the default value specified at the time of its creation.
- You can specify the default value directly when defining the column.

```
CREATE TABLE Employees (
    id INT,
    name VARCHAR(50),
    status VARCHAR(10) DEFAULT 'Active'
);
```

- Here, if no value is provided for status, it will automatically be 'Active'.

```
ALTER TABLE Employees
MODIFY status VARCHAR(10) DEFAULT 'Active';
```

- You can also change or remove the default value of an existing column.

- Change default:

```
ALTER TABLE Employees
ALTER COLUMN status SET DEFAULT 'Inactive';
```

- Remove default:

```
ALTER TABLE Employees
ALTER COLUMN status DROP DEFAULT;
```

- Check Constraint

- If we apply a CHECK constraint on any column, then whenever a user tries to insert a record into that column, the given condition will be evaluated. If the condition is true, the data will be inserted; otherwise, the insertion will fail.
- Create table with CHECK(column level)

```
CREATE TABLE Employee (
    empID INT PRIMARY KEY,
    name VARCHAR(50),
    age INT CHECK (age >= 18),
    salary DECIMAL(10,2) CHECK (salary > 0)
);
```

Here: age must be at least and salary must be greater than 0.

- Create table with CHECK (Table level)



```
CREATE TABLE Employee (
    empID INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    salary DECIMAL(10,2),
    CONSTRAINT chk_age CHECK (age >= 18),
    CONSTRAINT chk_salary CHECK (salary > 0)
);
```

Here we gave names chk_age and chk_salary to constraint.

- Add CHECK later using ALTER TABLE

```
ALTER TABLE Employee
ADD CONSTRAINT chk_age CHECK (age >= 18);
```

```
ALTER TABLE Employee
ADD CONSTRAINT chk_salary CHECK (salary > 0);
```

- Dropping CHECK constraint

```
ALTER TABLE Employee
DROP CHECK chk_age;
```

```
ALTER TABLE Employee
DROP CHECK chk_salary;
```

- Important Note

- If you don't name the constraint, MySQL auto-generates a name like employee_chk_1.
You can find it using:

- Not Null Constraint

- The NOT NULL constraint ensures that a column must always have a value — it cannot store NULL (missing) data.

- It's like marking a field in a form as "required" — you cannot leave it blank.
- We use it to maintain data integrity.

Some data fields are essential for meaningful records - for example:

- Every employee must have an Employee ID.
- Every student must have a Name.
- By applying NOT NULL, we make sure these important details are never left empty.

- Applying NOT NULL Constraint(while creating a table)

```
CREATE TABLE Employee (
    empID INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(25) NOT NULL,
    dept_ID VARCHAR(30)
);
```

- Here, name and email must have values when inserting data.
Department can be left NULL.

- Applying NOT NULL Constraint(after creating a table)

```
ALTER TABLE Employee
MODIFY name VARCHAR(50) NOT NULL;
```

- Dropping NOT NULL constraint

- You can remove the NOT NULL constraint if you want to allow NULL values.

```
ALTER TABLE Employee
MODIFY name VARCHAR(50) NULL;
```

Normalization

- Why Normalization

- Normalization is done to organize data in a database and to remove or reduce anomalies such as Insertion, Update, and Deletion anomalies. It helps in minimizing data redundancy, maintaining data consistency, and improving data integrity.
- For example, we have a table called encounter_record that shows which student enrolled in which course and which trainer is assigned. But right now, the details of students, courses, and trainers are all in the same table.

enrollment_record						
student_id	student_name	course_id	course_name	trainer_id	trainer_name	salary
1	Ram	1	C Programming	1	Rahul	25000
2	Rajesh	2	CPP	2	Sachin	30000
3	Gopal	1	C Programming	1	Rahul	25000
4	Shyam	1	C Programming	1	Rahul	25000
5	Pooja	3	Java	3	Suresh	35000
6	Priya	1	C Programming	1	Rahul	25000

- Here, we may face anomalies during insertion, updating, or deletion; an anomaly is a problem that occurs in specific situations. Let's see how
 - For example, if Rajesh decides to withdraw his admission, we would delete his record. Do you see any problem? Actually, there is one! The record for the CPP course and its trainer also gets deleted. This is called a Deletion anomaly.
 - Now, if we decide to increase Rahul's salary by 5000, we would update his record. Do you see any problem? Actually, there is one! Because Rahul's salary is repeated in multiple rows, the update will be applied to all rows, not just one. If the table had thousands of rows with repeated trainer information, the DBMS would have to update all occurrences, which takes more time and slows down the database. For now, it won't be noticeable because the data is small. This problem occurs during updating and is called an Updating anomaly.
 - Now, we are introducing a new course of JavaScript with a trainer named Hitesh and salary of 40000. Is it a problem? Yes, it is. According to our current table structure, we also need to add student details. But at this point, it's not necessary to have a student while adding course details. You might think of adding dummy values, but that can create problems in the future. This problem occurs during insertion and is called an Insertion anomaly.
- Types of normalization
 - 1NF
 - 2NF
 - 3NF

- BCNF
- 1st Normal Form
 - To make a table follow 1NF, each cell should contain only one value (atomic value).
 - In simple words:
 - Each column should hold a single value.
 - Should not have multivalued attributes (a column should not store more than one value).

studentID	studentName	courseName
1	Rajesh	C,C++
2	Ram	C

→

studentID	studentName	courseName
1	Rajesh	C
1	Rajesh	C++
2	Ram	C

- Explanation:
In the first table, the courseName column contains multiple values (C, C++).
In 1NF, we split these values into separate rows so that each cell has only one value.

This makes the data atomic and avoids multivalued attributes.

- Advantages of 1NF:
 - Makes data simpler and cleaner.
 - Removes repeating groups from the table.
 - Improves data consistency and reduces redundancy.
 - Easier to query and manage data.
- 2nd Normal Form
 - A table is said to be in Second Normal Form (2NF) if:
 - The table is already in 1NF, and
 - There is no partial dependency.

- Partial Dependency:

A partial dependency occurs when a non-prime attribute (a column that is not part of the primary key) depends on only a part of a candidate key key, instead of the whole key.

studentID	courseID	studentName	courseName	trainerName
1	1	Rajesh	C	Rahul
2	1	Ram	C	Rahul
3	2	Sanjay	C++	Hitesh

- Primary Key: (studentID, courseID) — Composite key
- Problem:

Here,

- studentName depends only on studentID.
 - courseName and trainerName depend only on courseID.
- So, some non-key columns depend on part of the primary key → Partial Dependency → Not in 2NF.

- After converting to 2NF:

Student Table

studentID	studentName
1	Rajesh
2	Ram
3	Sanjay

Course Table

courseID	courseName	trainerName
1	C	Rahul
2	C++	Hitesh

Enrolment Table

studentID	courseID
1	1
2	1
3	2

○ Explanation:

We removed partial dependency by dividing the table into smaller ones so that each non-key attribute depends on the whole primary key and not just a part of it.

○ Advantages of 2NF:

- Removes partial dependency.
- Reduces data redundancy.
- Makes data more consistent and structured.
- Simplifies data maintenance and updates.

● 3rd Normal Form

○ Definition:

A table is said to be in Third Normal Form (3NF) if:

- The table is already in 2NF, and
- There is no transitive dependency.

○ Transitive Dependency:

A transitive dependency occurs when a non-primary attribute depends on another non-primary attribute, instead of depending directly on the primary key.

In simple words,

- Every non-primary column should depend only on the primary key, not on another non-primary column.
- Example (Not in 3NF):
- Primary Key $\rightarrow$ ProductID

product_id	product_name	product_code	dealer_id	dealer_name
1	Monitor	M101	1	Rahul Traders
2	Keyboard	K202	2	Tech World

- Explanation:
 - product_name $\rightarrow$ depends on product_id (which is ok)
 - product_code $\rightarrow$ depends on product_id (which is ok)
 - But dealer_name $\rightarrow$ depends on dealer_id, not directly on product_id (which is not ok)
 - $\rightarrow$ This is a transitive dependency (since dealer_name depends on another non-key column).
- Hence, the table is not in 3NF.
- After converting to 3NF

Product Table

product_id	product_name	product_code	dealer_id
1	Monitor	M101	1
2	Keyboard	K202	2

Dealer Table

dealer_id	dealer_name
1	Rahul Traders
2	Tech World

- Explanation:
 - We removed the transitive dependency by creating a separate dealer table.
 - Now, all non-primary columns depend directly on the primary key (product_id) and not on another non-primary column.
- Advantages of 3NF:
 - Removes transitive dependency.
 - Eliminates data redundancy.
 - Ensures data integrity.
 - Makes the database more efficient and organized
- Boyce-Codd Normal Form(BCNF)
 - Definition:
 - A table is said to be in BCNF if:
 - The table is already in Third Normal Form (3NF), and
 - For every functional dependency ($X \rightarrow Y$), X is a superkey.

- In simple words,
Every determinant must be a candidate key.
- Key Concept:
 - A determinant is any attribute on which some other attribute is fully functionally dependent.
 - BCNF handles certain anomalies that 3NF cannot handle when there are multiple candidate keys.
- Example (Car Rentals Not in BCNF):

rental_id	car_plate	renter_name	renter_license	rental_fee
1	KA01AB1234	Rahul Sharma	L12345	500
2	KA02XY5678	Priya Singh	L67890	600
3	KA03CD9876	Rahul Sharma	L12345	700

- Explanation:
- Step 1: Identify the super key
 - **rental_id** alone can uniquely identify a row → super key
 - **car_plate** alone may not uniquely identify all rows if the same car is rented multiple times
 - **renter_license** alone cannot identify all rows because one renter can rent multiple cars
- Step 2: Functional Dependencies

Functional Dependency	Explanation
rental_id → car_plate, renter_license, rental_fee	Makes sense, rental_id is unique
car_plate → rental_fee	If rental fee depends only on the car (hypothetical case)
renter_license → renter_name	License uniquely identifies the renter's name

- Step 3: Check BCNF Rule
 - Rule: In $X \rightarrow Y$, X must be a super key.

Dependency	Super Key?	BCNF Status
rental_id → car_plate, renter_license, rental_fee	Yes	✓ Fine
car_plate → rental_fee	Not always (if car rented multiple times)	✗ Could violate BCNF
renter_license → renter_name	No	✗ Violation!
Explanation	renter_license is not a super key because a renter may have multiple rentals. Yet it determines renter_name.	

- Step 4: Fix BCNF Violation
 - Splitt tables

○ Car_Rentals Table

rental_id	car_plate	renter_id	rental_fee
1	KA01AB1234	1	500
2	KA02XY5678	2	600
3	KA03CD9876	1	700

○ Renter Table

renter_id	renter_license	renter_name
1	L12345	Rahul Sharma
2	L67890	Priya Singh

- Now, all functional dependencies are on **super keys**, and BCNF is satisfied.
- Advantages:
 - Removes BCNF violations by ensuring all determinants are super keys.
 - Reduces redundancy.
 - Maintains strict data integrity.
 - Easier to maintain and join tables in queries.

FirstBitSolutions.com
... Learn IT, Bit by Bit ...

Data Definition Language (DDL)

- Definition:
 - DDL stands for Data Definition Language.
It is a category of SQL commands used to define, modify, and delete database structures, such as tables, schemas, indexes, and constraints.
- DDL Commands
 - Create
 - Alter
 - Drop
 - Truncate
 - Rename
- Create command:
 - We use the CREATE command in MySQL to create **DATABASES**, **TABLES**, **VIEWS**, **INDEXES**, and _____.
 - To create a new database:

CREATE **DATABASE** _____ **employee_db;**

- To use / activate a database

USE _____ **employee_db;**

- To see the databases or tables

SHOW _____ **DATABASES;**

SHOW TABLES;

- To see table structure

DESC **employee;**

- To create a table

CREATE TABLE _____ **employee (id INT PRIMARY KEY,**
name VARCHAR(50) NOT NULL,
salary DECIMAL(10,2),
dept_id INT
);

- MySQL Data Types

In MySQL, each column in a table is defined with a **data type**, which tells what kind of value it can store.

- Numeric Data Types

Type	Description	Example
INT	Whole numbers	10, -45
BIGINT	Very large integers	9876543210
DECIMAL (p, s)	Fixed-point numbers (precision p, scale s)	12.50
FLOAT	Floating-point numbers (approx.)	10.25
DOUBLE	Double-precision floating number	123.4567

- String (Character) Data Types

Type	Description	Example
DATE	Stores date only	'2025-10-27'
TIME	Stores time only	'14:25:00'
DATETIME	Stores both date and time	'2025-10-27 14:25:00'
YEAR	Stores year value	2025

- Boolean Data Type

Type	Description	Example
BOOLEAN	TRUE (1) or FALSE (0)	1

- Alter command

- The ALTER command in MySQL is used to modify the structure of an existing table.
- It allows you to add, modify, or delete columns and constraints.
- Add a new column at the end

ALTER TABLE employee ADD COLUMN age INT;

- Add a new column at the beginning

ALTER TABLE employee ADD COLUMN doj DATE FIRST;

- Add a new column after a specific column

ALTER TABLE employee ADD COLUMN city VARCHAR(30) AFTER name;

- Change the data type of a column

ALTER TABLE employee MODIFY COLUMN salary DECIMAL(7,2);

- Change the position of a column

ALTER TABLE employee MODIFY COLUMN id INT FIRST;

- Rename a Column

ALTER TABLE employee RENAME COLUMN name TO emp_name;

- Add a **Primary Key**

ALTER TABLE employee ADD PRIMARY KEY (id);

- Add a **Check Constraint**

ALTER TABLE employee ADD CONSTRAINT chk_age CHECK (age >= 18);

- Drop a **Primary Key**

ALTER TABLE employee DROP PRIMARY KEY;

- Drop a **Check Constraint**

ALTER TABLE employee DROP CONSTRAINT chk_age;

- Drop a **Column**

ALTER TABLE employee DROP COLUMN city;

- **Summary:**

The ALTER command helps to change the structure of an existing table without deleting it. It's one of the most frequently used DDL commands during database updates.

- **DROP command**

- The DROP command in MySQL is used to permanently delete database objects such as DATABASES, TABLES, VIEWS, or INDEXES.
- Once dropped, the data and structure cannot be recovered (unless backed up).
- Removes a specific column from a table.

ALTER TABLE employee DROP COLUMN age;

- Removes the primary key constraint from the table.

ALTER TABLE employee DROP PRIMARY KEY;

- Removes a named constraint such as a CHECK or UNIQUE

ALTER TABLE employee DROP CONSTRAINT chk_age;

- Deletes the table and all its data permanently.

DROP TABLE employee;

- Deletes the entire database and all the tables within it.

DROP DATABASE employee_db;

- Summary
 - DROP is irreversible — it removes the structure and data permanently.
 - Always double-check before using it, especially in production databases.
- TRUNCATE Command
 - The TRUNCATE command in MySQL is used to remove all rows from a table but keep the table structure intact.
 - It is faster than DELETE because it does not log each row deletion individually.
 - After truncating, the table is empty but can still be used to insert new records.
 - Syntax:

TRUNCATE TABLE table_name;
 - Example:

TRUNCATE TABLE employee;
 - This will remove all data from the employee table but keep the columns (id, emp_name, salary, dept_id) and constraints (like primary key).
 - Key Points:
 - It cannot be rolled back once executed (unless used inside a transaction in some DB engines).
 - It resets AUTO_INCREMENT values to zero.
 - You cannot use WHERE with TRUNCATE.
 - Table structure and constraints remain unchanged.
 - Summary:

Use TRUNCATE when you want to quickly clear all data from a table but still keep its structure for future use.
- RENAME Command
 - The RENAME command in MySQL is used to change the name of a database object, such as a table or database.
 - It helps to rename objects without losing data or structure.
 - You can rename an existing table using either of the following syntaxes:
 - Syntax 1:

RENAME TABLE old_table_name TO new_table_name;
 - Example:

RENAME TABLE employee TO emp_master;

 The table `employee` is now renamed to `emp_master` with all its data and structure preserved.
 - Rename Multiple Tables at Once



RENAME TABLE employee TO emp_master, department TO dept_master;

- Rename a Database (Using MOVE)
- MySQL doesn't have a direct command to rename a database, but you can achieve it using RENAME TABLE along with the database name.
- Example:

RENAME TABLE old_db.employee TO new_db.employee;

This effectively moves the table from old_db to new_db.

- Rename a Column (MySQL 8.0.13 and above)

ALTER TABLE emp_master RENAME COLUMN emp_name TO employee_name;

FirstBitSolutions.com
... Learn IT, Bit by Bit ...

Data Manipulation Language(DML)

- Definition:
 - DML is a category of SQL commands used to manipulate the data stored in database tables.
 - Unlike DDL, DML does not change the structure of tables — it only affects the **data**.
- Main DML Commands
 - INSERT → To add new records (rows) into a table.
 - UPDATE → To modify existing records.
 - DELETE → To remove records from a table.

id	emp_name	salary	dept_id
1	Rohit Roy	50000	101
2	Priya Singh	60000	102
3	Amit Kumar	55000	101

- INSERT Command
 - Used to add new data into a table.
 - Syntax 1 (Insert values into all columns):
INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
 - Example:
INSERT INTO employee (id, emp_name, salary, dept_id) VALUES (4, 'Sneha Patil', 58000, 103);
 - Syntax 2 (Insert values into specific columns only):
If you want to insert data only into a few columns, you can skip the others (they will take NULL or default values).
INSERT INTO employee (id, emp_name) VALUES (5, 'Rohit Jadhav');
Only id and emp_name get values;
salary and dept_id remain NULL (or default values if defined).
 - Syntax 3 (Insert multiple rows at once):
INSERT INTO employee (id, emp_name, salary, dept_id) VALUES (6, 'Pooja Deshmukh', 62000, 102), (7, 'Karan Shah', 58000, 101);
Adds multiple records in a single query — faster and cleaner.
- WHERE Clause (Basic Introduction)
 - Purpose:
 - The WHERE clause is used to filter rows in a SQL statement — it decides which rows are affected by UPDATE, DELETE, or SELECT.
 - Basic Syntax:
**UPDATE table_name
SET column_name = new_value
WHERE condition;**

**DELETE FROM table_name
WHERE condition;**
 - Why WHERE Clause Matters



- If you omit the WHERE clause, the command applies to every record in the table.
 - Always double-check your WHERE condition before executing UPDATE or DELETE.
- **UPDATE Command**
 - The UPDATE command is used to modify existing values in a table based on certain conditions, which are applied using the WHERE clause.
 - Syntax:
UPDATE table_name
SET column_name = new_value
WHERE condition;
 - Example 1 – Update single column
 - Let's say we want to increase the salary of employee with id = 3.
UPDATE employee
SET salary = 55000
WHERE id = 3;
 - Explanation
 - UPDATE employee → choose the table
 - SET salary = 55000 → change value
 - WHERE id = 3; → only update that one row
 - Example 2 – Update multiple columns
UPDATE employee
SET emp_name = 'Rohit Sharma', salary = 60000
WHERE id = 2;
 Updates both emp_name and salary for employee with id = 2.
 - Example 3 – Update all rows (without WHERE)
UPDATE employee
SET dept_id = 101;
 Warning: This updates every row in the table.
 That's why the WHERE clause is very important, it helps you control which rows get updated.
- **Delete Command**
 - The DELETE command is used to remove specific records or multiple records from a table based on certain conditions, which are applied using the WHERE clause.
 - Basic Syntax:
DELETE FROM table_name
WHERE condition;
 - Example 1 – Delete a specific record
DELETE FROM employee
WHERE id = 4;
 Removes only the row where id = 4.
 - Example 2 – Delete all rows (without WHERE)
DELETE FROM employee;
 Warning: This deletes all records from the table, but the table structure remains.

- Difference Between DROP, DELETE, and TRUNCATE

These three SQL commands are used to **remove data**, but they work in **different ways** and have **different effects** on tables.

Feature	DELETE	TRUNCATE	DROP
Type			
Removes	Specific rows	All rows	Table structure + data
WHERE Clause	Allowed	Not allowed	Not allowed
Table Structure Remains			
Can Rollback	Yes	No	No
Speed	Slow (row by row)	Faster than DELETE	Fastest
Auto Increment Reset			
Example	DELETE FROM employee WHERE salary < 30000;	TRUNCATE TABLE employee;	DROP TABLE employee;
Use When	You want to remove some data	You want to clear the table quickly	You no longer need the table

DQL (Data Query Language)

- Definition:

- It is used to **fetch** or **retrieve** data from a database.
- The main command under DQL is the SELECT statement.

- SELECT

- Used to fetch data from one or more tables based on specific conditions, which are applied using the WHERE clause.
- Syntax:

SELECT column1, column2, ...

FROM table_name

WHERE condition;

- Example:

SELECT emp_name, salary FROM employee;

- Variations of SELECT Queries (Basic)

Type	Description	Example
SELECT *	Displays _____ columns	SELECT * FROM employee;
SELECT specific columns	Displays _____ columns	SELECT emp_name, salary FROM employee;
SELECT with WHERE	Retrieves specific rows based on _____	SELECT * FROM employee WHERE dept_id = 2;

- WHERE Clause with Operators

Operator	Description	Example
=	Equal to	salary = 30000
!= or <>	Not equal to	dept_id != 2
>	Greater than	salary > 25000
<	Less than	salary < 40000
>=	Greater than or equal to	salary >= 25000
<=	Less than or equal to	salary <= 40000
AND	Both conditions must be true	dept_id = 2 AND salary > 25000
OR	Any one condition must be true	dept_id = 2 OR salary > 25000
BETWEEN	Range condition	salary BETWEEN 20000 AND 40000
IN	Matches any value in a list	dept_id IN (1, 2, 3)
IS NULL	Checks for NULL values	dept_id IS NULL

- Example:

-- Display all employees

SELECT * FROM employee;

-- Display employee names and salaries

SELECT emp_name, salary FROM employee;

-- Display employees whose salary is greater than 30000

SELECT * FROM employee WHERE salary > 30000;

-- Display employees in department 1 or 2

SELECT * FROM employee WHERE dept_id IN (1, 2);

Pattern Matching (LIKE Operator)

- Definition:

- The LIKE operator is used to search for a specific pattern in a column.
- It is commonly used with the WHERE clause.
- Patterns are created using wildcard characters.

- Wildcards Used with LIKE

Wildcard	Description	Example	Matches
%	Represents zero or more characters	emp_name LIKE 'A%'	Names starting with A (e.g., <i>Anuj</i> , <i>Akash</i> , <i>Amit</i>)
_	Represents a single character	emp_name LIKE '_nuj'	Names like <i>Anuj</i> (second letter 'n')

- Syntax:

```
SELECT column1, column2  
FROM table_name  
WHERE column_name LIKE 'pattern';
```

```
-- Employees whose name starts with 'A'  
SELECT * FROM employee WHERE emp_name LIKE 'A%';
```

```
-- Employees whose name ends with 'a'  
SELECT * FROM employee WHERE emp_name LIKE '%a';
```

```
-- Employees whose name contains 'it'  
SELECT * FROM employee WHERE emp_name LIKE '%it%';
```

```
-- Employees whose name has 'n' as the second character  
SELECT * FROM employee WHERE emp_name LIKE '_n%';
```

- Note
 - The LIKE operator is case-insensitive in MySQL by default.
 - You can also use NOT LIKE to exclude specific patterns.
 - Example:
 - **SELECT * FROM employee WHERE emp_name NOT LIKE 'A%';**

DISTINCT Keyword

- Why do we need DISTINCT?
When we query data using SELECT, sometimes the result contains duplicate values.
If we want to display unique (non-repeated) records only, we use the DISTINCT keyword.
- Definition
 - The DISTINCT keyword is used with SELECT to eliminate duplicate rows from the result set.
 - It ensures that each row in the output is unique.
- Syntax
SELECT DISTINCT column1, column2, ...
FROM table_name;
- Examples
 - Retrieve unique department IDs
SELECT DISTINCT dept_id
FROM employee;
Returns each dept_id only once — even if multiple employees belong to the same department.
 - Retrieve unique combinations of department and salary
SELECT DISTINCT dept_id, salary
FROM employee;
Returns unique pairs of dept_id and salary.
 - Compare with normal SELECT
SELECT dept_id FROM employee;



May return duplicate department IDs.

SELECT DISTINCT dept_id FROM employee;

Removes duplicates and returns only unique ones.

- Key Points
 - DISTINCT applies to all selected columns together, not individually.
 - It can be used with functions like COUNT() to count unique values:
SELECT COUNT(DISTINCT dept_id) AS total_departments
FROM employee;
 - Using DISTINCT on large datasets can affect performance (since duplicates must be checked).

FirstBitSolutions.com
... Learn IT, Bit by Bit ...

ORDER BY Clause

- Definition:
 - The ORDER BY clause is used to _____ the result set of a query.
 - It can sort the data in ascending (ASC) or descending (DESC) order.
 - By default, sorting is done in ascending order.
- Syntax:


```
SELECT column1, column2
FROM table_name
ORDER BY column_name [ASC | DESC];
```
- Examples:


```
-- Display employees in ascending order of salary
SELECT * FROM employee
ORDER BY salary;

-- Display employees in descending order of salary
SELECT * FROM employee
ORDER BY salary DESC;

-- Display employees sorted by department and then by name
SELECT * FROM employee
ORDER BY department, emp_name;
```
- Notes
 - You can sort the result by more than one column.
 - When sorting by multiple columns:
 - The first column decides the main order.
 - The next column is used when there is a tie in the first.
 - ASC (ascending) is optional; DESC must be mentioned explicitly.
- Tell me students:

Now that we know how to arrange our data using ORDER BY, what if we don't want to display all the sorted records, and only need the top 5 or first 10 results?

That's where the LIMIT clause comes in.
- LIMIT Clause
 - The LIMIT clause is used to restrict the number of rows returned by a query.
 - It helps when we only need a few records, for example, top 5 salaries or first 10 employees.
 - It is commonly used with ORDER BY for predictable output.
 - Syntax:


```
SELECT column1, column2
FROM table_name
ORDER BY column_name [ASC | DESC]
LIMIT number_of_records;
```
 - Examples
 - Retrieve top 5 employees with highest salary

- Retrieve first 3 employees by id
**SELECT * FROM employee
ORDER BY id ASC
LIMIT 3;**

Now what if we want to skip the first few records and then display the next set?

That's where the OFFSET clause is useful.

- **OFFSET**

- The OFFSET clause skips a specific number of rows before starting to return results.
- It is mostly used along with LIMIT for pagination or partial data viewing.
- Syntax
**SELECT column1, column2
FROM table_name
ORDER BY column_name [ASC | DESC]
LIMIT number_of_records OFFSET number_of_rows_to_skip;**
- Example
**SELECT * FROM employee
ORDER BY salary DESC
LIMIT 5 OFFSET 5;**
- Pagination Concept using LIMIT and OFFSET
When displaying records in multiple pages (like in a website or app), we use LIMIT and OFFSET together to control which rows appear on each page.

Page No.	Rows to Display	LIMIT	OFFSET	Query Example
Page 1	1–5	5	0	SELECT * FROM employee ORDER BY id ASC LIMIT 5 OFFSET 0;
Page 2	6–10	5	5	SELECT * FROM employee ORDER BY id ASC LIMIT 5 OFFSET 5;
Page 3	11–15	5	10	SELECT * FROM employee ORDER BY id ASC LIMIT 5 OFFSET 10;
Page 4	16–20	5	15	SELECT * FROM employee ORDER BY id ASC LIMIT 5 OFFSET 15;

- Explanation:
 - LIMIT → decides how many rows to show.
 - OFFSET → decides from where to start showing rows.
 - Together, they help create a smooth pagination system for user-friendly data display.

Aggregate Functions

- Why do we need Aggregate Functions?
 - So far, we have used SQL to retrieve or filter individual rows using SELECT and WHERE.
But what if we want summary information — for example:

- How many employees are there in total?
- What is the highest salary?
- What is the average salary per department?

For such summary-level or group-based calculations, we use Aggregate Functions.

- Definition
 - Aggregate functions perform a calculation on a set of values and return a single summarized value.
 - They are used with the SELECT statement, often along with GROUP BY.
 - Commonly used to find totals, averages, counts, minimum, and maximum values.
- Example for each function
 - Count total employees
SELECT _____() AS total_employees
FROM employee;
 Returns total number of rows in the employee table.
 - Find total salary
SELECT _____(salary) AS total_salary
FROM employee;
 Calculates the sum of all employees' salaries.
 - Find average salary
SELECT _____() AS average_salary
FROM employee;
 Returns the average salary of employees.
 - Find highest and lowest salary
SELECT _____(salary) AS _____salary, _____(salary)
AS _____salary
FROM employee;
 Returns both maximum and minimum salaries in a single query.
 - Count employees in a specific department
SELECT COUNT(*) AS total_in_sales
FROM employee
WHERE dept_id = 101;
 Counts only those employees whose dept_id is 101.
- Key Points
 - NULL values are ignored in aggregate functions (except COUNT(*)).
 - You can use aliases (AS) to rename output columns for better readability.
 - Aggregate functions can be used with GROUP BY to summarize data by groups.

GROUP BY Clause

- Why do we need GROUP BY?
 - After learning aggregate functions, we can calculate totals, averages, or counts, but what if we want these summaries for each department, for each city, or for each product?
 - To perform aggregate calculations group-wise, we use the GROUP BY clause.
- Definition
 - The GROUP BY clause is used to group rows that have the same values in one or more columns.
 - It is often used with aggregate functions (COUNT(), SUM(), AVG(), etc.) to summarize data for each group.
- How GROUP BY Works (Step-by-Step)
 - SELECT data from the table.
 - Divide rows into groups based on the column(s) mentioned in GROUP BY.
 - Apply aggregate functions (like COUNT(), SUM(), AVG()) within each group.
 - Return one row per group with the summarized result.
- Analogy: Sorting into Buckets

Imagine you have a box full of employee slips with dept_id written on each.

 - Step 1: **Sort** slips by dept_id.
 - Step 2: **Put** slips with the same dept_id into the **same bucket**.
 - Step 3: **Calculate** totals or averages for each bucket.

Each bucket → one group
Label on the bucket → column in GROUP BY
Total or average → aggregate result
- Example Illustration

emp_id	emp_name	salary	dept_id
1	Amit	40000	101
2	Raj	35000	101
3	Neha	30000	102
4	Ravi	25000	102

→ **GROUP BY dept_id**

Buckets formed:

- Dept 101 → (Amit – 40000, Raj – 35000) → SUM = 75000
- Dept 102 → (Neha – 30000, Ravi – 25000) → SUM = 55000

- Output:

dept_id	total_salary
101	75000
102	55000

- Syntax


```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

- Examples

- Count employees in each department

```
SELECT dept_id, COUNT(*) AS total_employees  
FROM employee  
GROUP BY dept_id;
```

Output:

dept_id	total_employees
101	2
102	2

- Total Salary per Department

```
SELECT dept_id, SUM(salary) AS total_salary  
FROM employee  
GROUP BY dept_id;
```

Output:

dept_id	total_salary
101	75000
102	55000

- Using Multiple Columns in GROUP BY

- We can group data based on more than one column.

```
SELECT dept_id, emp_name, SUM(salary)  
FROM employee  
GROUP BY dept_id, emp_name;
```

Here, MySQL first groups by dept_id, and then by emp_name within each department.

- Combining GROUP BY with ORDER BY

- We can also **sort** the grouped result.

```
SELECT dept_id, SUM(salary) AS total_salary  
FROM employee  
GROUP BY dept_id  
ORDER BY total_salary DESC;
```

After grouping, results are sorted by the total_salary in descending order.

- Filtering Groups using HAVING

- Sometimes, you may want to filter grouped results — for example, show only those departments where the total salary is more than ₹60,000.

- For that, we use the HAVING clause.

- Remember:

- WHERE → filters rows before grouping
- HAVING → filters groups after grouping

- Syntax:

```
SELECT dept_id, SUM(salary) AS total_salary  
FROM employee  
GROUP BY dept_id  
HAVING total_salary > 60000;
```

Output:

dept_id	total_salary
101	75000

- **Quick Tip:** You can use `WHERE` and `HAVING` together:

SELECT dept_id, SUM(salary)

FROM employee

WHERE salary > 25000

GROUP BY dept_id

HAVING SUM(salary) > 60000;

Explanation:

- `WHERE salary > 25000` → filters out employees earning 25,000 or less before grouping.
- `GROUP BY dept_id` → groups remaining employees department-wise.
- `HAVING SUM(salary) > 60000` → shows only those departments whose total salary (after grouping) exceeds 60,000.
- **Important Rule While Using GROUP BY (Must Know)**
 - When you use the **GROUP BY** clause:
 - Every column in the `SELECT` list must either
 - ➔ be part of the `GROUP BY` clause **OR**
 - ➔ be used inside an **aggregate function** (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`).
 - You **CANNOT** select a column that is neither grouped nor aggregated.
 - Invalid Example (Violates `GROUP BY` Rule)


```
SELECT dept_id, emp_name, SUM(salary)
FROM employee
GROUP BY dept_id;
```
 - MySQL (with strict SQL mode) will throw an error:
Expression #2 of SELECT list is not in GROUP BY clause and contains nonaggregated column

Built-in (Scalar) Functions

Till now, we have learned how to summarize and analyze data using aggregate functions along with GROUP BY and HAVING.

These functions work on a group of rows and return one result per group.

But sometimes, we need to perform operations on each individual value — like changing text case, finding string length, rounding numbers, or getting the current date.

- Definition:
 - Built-in functions are predefined MySQL functions that perform operations on data and return one value per row.
 - They help in text formatting, mathematical calculations, date handling, and system info retrieval.

- String Functions

String functions are used to perform operations on text data such as names, cities, or departments.

They help us manipulate, extract, or compare character data easily.

- Commonly Used String Functions

-- Convert to uppercase

SELECT UPPER('ramesh');

-- Output: RAMESH

-- Convert to lowercase

SELECT LOWER('MUMBAI');

-- Output: mumbai

-- Find string length

SELECT LENGTH('Mahesh');

-- Output: 6

-- Join two strings

SELECT CONCAT('Ramesh', '-', 'Mumbai');

-- Output: Ramesh-Mumbai

-- Extract substring (from 2nd position, length 3)

SELECT SUBSTRING('Ramesh', 2, 3);

-- Output: ame

-- Return first 3 characters

SELECT LEFT('Ramesh', 3);

-- Output: Ram

-- Return last 3 characters

SELECT RIGHT('Ramesh', 3);

-- Output: esh

-- Remove spaces from both sides

SELECT TRIM(' Ramesh ');

-- Output: Ramesh

-- Replace substring
SELECT REPLACE('Mumbai', 'Mumbai', 'Delhi');
-- Output: Delhi

-- Reverse a string
SELECT REVERSE('Ramesh');
-- Output: hsemaR

-- Find position of substring
SELECT INSTR('Ramesh', 'sh');
-- Output: 5

-- Find position of substring (alternative to INSTR)
SELECT LOCATE('a', 'Ramesh');
-- Output: 2

-- Add characters to the left to make length 5
SELECT LPAD('25', 5, '0');
-- Output: 00025

-- Add characters to the right to make length 5
SELECT RPAD('25', 5, '0');
-- Output: 25000

-- Find position of value in list
SELECT FIELD('B', 'A', 'B', 'C');
-- Output: 2

○ Summary

String functions are helpful for:

- Cleaning and formatting text data (TRIM, UPPER, LOWER)
- Extracting and modifying parts of strings (SUBSTRING, REPLACE, LEFT, RIGHT)
- Searching and analyzing text (INSTR, LOCATE, REVERSE)
- Formatting values with padding or position-based lookups (LPAD, RPAD, FIELD)

● Numeric Functions

Numeric functions are used to perform _____ operations on numeric data such as salary, marks, or quantity.

They help us calculate, round, compare, or generate numeric values easily.

○ Commonly Used String Functions

-- Return absolute (positive) value

SELECT ABS(-25);

-- Output: 25

-- Return the largest integer less than or equal to value

SELECT FLOOR(15.75);

-- Output: 15

-- Return the smallest integer greater than or equal to value

SELECT CEIL(15.75);



-- Output: 16

-- Round to nearest integer
SELECT ROUND(15.49);

-- Output: 15

-- Round to 2 decimal places
SELECT ROUND(15.5678, 2);

-- Output: 15.57

-- Truncate decimal part after 2 digits (no rounding)
SELECT TRUNCATE(15.5678, 2);

-- Output: 15.56

-- Return remainder of division
SELECT MOD(17, 5);

-- Output: 2

-- Generate random number between 0 and 1
SELECT RAND();

-- Output: 0.32456789 (varies every time)

-- Return square root
SELECT SQRT(81);

-- Output: 9

-- Return power of a number
SELECT POW(2, 4);

-- Output: 16

-- Return greatest value among list
SELECT GREATEST(25, 48, 12, 55);

-- Output: 55

-- Return smallest value among list
SELECT LEAST(25, 48, 12, 55);

-- Output: 12

-- Sign of number (-1, 0, or 1)
SELECT SIGN(-20), SIGN(0), SIGN(30);

-- Output: -1 | 0 | 1

○ Summary

Numeric functions are used for mathematical calculations and number formatting. They help in rounding, generating random numbers, finding powers, and identifying minimum or maximum values.

Commonly used functions include:

ABS, FLOOR, CEIL, ROUND, TRUNCATE, MOD, RAND, SQRT, POW, GREATEST, LEAST, and SIGN.

- Date and Time Functions

Date and time functions are used to handle and manipulate date or time values such as joining date, order date, or payment time.

They help us extract parts of a date, perform calculations, and format date/time values easily.

- Commonly Used String Functions

-- Return current date

SELECT CURDATE();

-- Output: 2025-10-27

-- Return current time

SELECT CURTIME();

-- Output: 21:30:45

-- Return current date and time

SELECT NOW();

-- Output: 2025-10-27 21:30:45

-- Return current year, month, and day

SELECT YEAR(NOW()), MONTH(NOW()), DAY(NOW());

-- Output: 2025 | 10 | 27

-- Return name of the month

SELECT MONTHNAME('2025-10-27');

-- Output: October

-- Return name of the weekday

SELECT DAYNAME('2025-10-27');

-- Output: Monday

-- Extract specific parts of date

**SELECT EXTRACT(YEAR FROM '2025-10-27') AS Year,
EXTRACT(MONTH FROM '2025-10-27') AS Month,
EXTRACT(DAY FROM '2025-10-27') AS Day;**

-- Output: 2025 | 10 | 27

-- Add 10 days to date

SELECT DATE_ADD('2025-10-27', INTERVAL 10 DAY);

-- Output: 2025-11-06

-- Subtract 15 days from date

SELECT DATE_SUB('2025-10-27', INTERVAL 15 DAY);

-- Output: 2025-10-12

-- Find difference between two dates (in days)

SELECT DATEDIFF('2025-10-27', '2025-10-01');

-- Output: 26

-- Format date in a specific pattern

SELECT DATE_FORMAT('2025-10-27', '%d-%m-%Y');

-- Output: 27-10-2025

-- Return current timestamp

SELECT CURRENT_TIMESTAMP();

-- Output: 2025-10-27 21:30:45

-- Return only time from datetime

SELECT TIME(NOW());

-- Output: 21:30:45

-- Return hour, minute, second

SELECT HOUR(NOW()), MINUTE(NOW()), SECOND(NOW());

-- Output: 21 | 30 | 45

- Summary

Date and time functions are used to fetch the current date/time, extract specific parts, calculate intervals, or format output.

They are essential for operations involving logs, billing cycles, attendance, or time-based reports.

Commonly used functions include:

CURDATE, CURTIME, NOW, YEAR, MONTH, DAY, MONTHNAME, DAYNAME, DATE_ADD, DATE_SUB, DATEDIFF, DATE_FORMAT, and CURRENT_TIMESTAMP.

FirstBitSolutions.com
... Learn IT, Bit by Bit ...

Subquery

So far, we've learned how to:

- Retrieve specific columns using SELECT
- Filter data using WHERE
- Sort results using ORDER BY / LIMIT
- Summarize and group data using GROUP BY
- Perform calculations using functions

Now, what if we want to use the result of one query inside another query?

For example, find employees who earn more than the average salary.

- What is a Subquery?

A subquery is a _____ inside _____ query.

It is used to retrieve data that will be used by the main (_____) query.

The inner query executes first, and its result is passed to the outer query.

- Why We Use Subqueries

We use subqueries when:

- We need a **value computed by another query** (like AVG, COUNT, etc.).
- We want to **filter data dynamically**.
- We need to **simplify multi-step logic** into one query.
- We want to **compare a column value** with results from another query.
- In short if we want to compare the result of an aggregate function or another query with each and every row of our table.

- Examples:

- Example 1:

If you want to find employees who earn more than the average salary, you first need the average salary (from one query) and then compare it with each employee's salary — that's exactly where subqueries help.

- Example 2:

If you want to print all the employees who are working in the HR department, but the dept_name column is present in the department table, you can use a subquery to first find the department ID of the HR department, and then use that result to filter employees.

- Syntax

SELECT column_name

FROM table_name

WHERE column_name operator (SELECT column_name FROM table_name WHERE condition);

- Example

- Example 1: Employees earning more than average salary

SELECT emp_name, salary

FROM employee

WHERE salary > (SELECT AVG(salary) FROM employee);

Explanation:

Inner query → (SELECT AVG(salary) FROM employee) returns one value (average salary).

Outer query → compares each employee's salary with that value.

- Example 2: Employees working in HR department

SELECT emp_name

FROM employee

WHERE dept_id = (SELECT dept_id FROM department WHERE dept_name = 'HR');

Explanation:

The inner query retrieves the dept_id of the HR department.

The outer query displays all employees whose dept_id matches that value.

- Types of Subqueries

Subqueries can be classified based on the number of rows they return and how they are executed.

- Single-row Subquery

- Returns only one row as a result.

- Usually used with comparison operators like =, >, <, >=, <=.

Example:

Find employees who earn more than the average salary.

SELECT emp_name, salary

FROM employee

WHERE salary > (SELECT AVG(salary) FROM employee);

Explanation:

The inner query returns a single value (average salary),

and the outer query compares each employee's salary with that value.

- Multiple-row Subquery

- Returns more than one row.

- Requires special operators like IN, ANY, or ALL.

Example: Subquery with IN

Find employees who work in departments located in Pune.

SELECT emp_name

FROM employee

WHERE dept_id IN (SELECT dept_id FROM department WHERE dept_location = 'Pune');

Explanation:

The inner query may return multiple department IDs.

The outer query selects employees whose dept_id is in that list.

Example: Subquery with ANY

Compares a value to each value returned by the subquery and returns TRUE if the condition is satisfied by any *one* of them.

Example:

Find employees who earn more than any employee in department 10 (i.e., higher than at least one person in dept 10).

SELECT emp_name, salary

FROM employee

WHERE salary > ANY (SELECT salary FROM employee WHERE dept_id = 10);

Explanation:

The inner query returns salaries of employees in department 10.

The outer query selects employees whose salary is greater than *any one* of those salaries.

So even if one person in dept 10 earns 30,000, employees earning more than 30,000 will be included.

Example: Subquery with ALL

Compares a value to each value returned by the subquery and returns TRUE only if the condition is satisfied by all of them.

Example:

Find employees who earn more than all employees in department 10 (i.e., higher than the highest salary in dept 10).

```
SELECT emp_name, salary  
FROM employee  
WHERE salary > ALL (SELECT salary FROM employee WHERE  
dept_id = 10);
```

Explanation:

The inner query again returns all salaries from department 10.

The outer query selects employees whose salary is greater than every one of those salaries.

So only employees whose salary is greater than the maximum salary in dept 10 will appear.

- **Correlated Subquery**

- A correlated subquery depends on the outer query for its values.
- It executes once for every row of the outer query.
- It cannot run independently.

Example:

Find employees whose salary is greater than the average salary of their department.

```
SELECT emp_name, salary, dept_id  
FROM employee e  
WHERE salary > (SELECT AVG(salary) FROM employee WHERE  
dept_id = e.dept_id);
```

Explanation:

For each employee in the outer query, the inner query calculates the average salary of that employee's department.

The comparison is done row by row.

- **Summary**

Subqueries help us use one query's output as another query's input — useful when you want to filter or compare data across related tables or computed results.

JOINS

In subqueries, we compared data between tables indirectly, by running one query inside another.

But sometimes, we want to combine and fetch data from multiple tables at once, especially when they share a common column (like dept_id, emp_id, etc.).

That's where **Joins** come in.

- What is a Join?

A **Join** is used to combine rows from two or more tables based on a _____ column between them.

Joins help you view connected information that is stored in separate tables, for example:

- employee table → employee details
- department table → department details

If both tables share a dept_id column, we can join them to display each employee with their department name.

- Basic Syntax:

```
SELECT column_list
FROM table1
JOIN_TYPE table2
ON table1.common_column = table2.common_column;
```

- CROSS JOIN

- Why we use it

We use CROSS JOIN when we want to generate all possible combinations between two tables.

It is useful for “**what-if**” scenarios, where we want to test or compare every possible pairing of data.

For example I want to see all possible employee–department combinations before assigning departments.

- Syntax

```
SELECT column_list
FROM table1
CROSS JOIN table2;
```

- Example

employee table

emp_id	emp_name	salary	dept_id
1	Ramesh	45000	10
2	Suresh	55000	20

department table

dept_id	dept_name	dept_location
10	HR	Mumbai
20	IT	Pune

- Query

```
SELECT emp_name, salary, dept_name, dept_location
```

FROM employee CROSS JOIN department;

- Output

emp_name	salary	dept_name	dept_location
Ramesh	45000	HR	Mumbai
Ramesh	45000	IT	Pune
Suresh	55000	HR	Mumbai
Suresh	55000	IT	Pune

- Summary

- Produces all possible combinations of rows from both tables.
- Used for “what-if” or testing scenarios.
- Total rows = (rows in first table) × (rows in second table).
- No condition (ON or WHERE) is required.

• INNER JOIN

Why we use it

We use INNER JOIN when we want to combine related data from two tables based on a matching condition.

It returns only the rows where there is a match in both tables.

- Example use case:

To display employee names along with their department names and locations.

- Syntax

SELECT column_list

FROM table1

INNER JOIN table2

ON table1.common_column = table2.common_column;

You can also use **aliases** to make queries shorter and more readable:

SELECT alias.column_list

FROM table1 t1

INNER JOIN table2 t2

ON t1.common_column = t2.common_column;

- Example

employee table

emp_id	emp_name	salary	dept_id
1	Ramesh	45000	10
2	Suresh	55000	20
3	Mahesh	60000	30

department table

dept_id	dept_name	dept_location
10	HR	Mumbai
20	IT	Pune

- Query
SELECT e.emp_name, e.salary, d.dept_name, d.dept_location
FROM employee e
INNER JOIN department d
ON e.dept_id = d.dept_id;

- Output

emp_name	salary	dept_name	dept_location
Ramesh	45000	HR	Mumbai
Suresh	55000	IT	Pune

- Explanation
 - Only rows with matching dept_id in both tables are displayed.
 - Mahesh is not displayed, because dept_id = 30 has no match in the department table.
 - This is the most commonly used join, since we usually need only matching data from both tables.
 - Using aliases (e, d) makes long queries shorter and cleaner.
- Summary
 - Returns only matching rows from both tables.
 - Used when we need related data from multiple tables.
 - Most regularly used join, since we usually work with matching records only.
 - Supports aliases for better readability.

• Left Outer Join

- Why We Use It
 Left Outer Join is used when we want to include all records from the _____ table (usually the main or primary table), even if there are no matching records in the _____ table.
 It ensures that we don't lose any data from the left table while still displaying matching rows from the right table when available.
- Analogy:
 Imagine you have a list of all employees, and you want to see their departments.
 Even if some employees are not assigned to any department yet — you still want to show them in the result.
- Syntax
SELECT alias.column_list
FROM table1 t1
LEFT JOIN table2 t2
ON t1.common_column = t2.common_column;
- Example Tables
 employee table

emp_id	emp_name	salary	dept_id
1	Ramesh	45000	10

emp_id	emp_name	salary	dept_id
2	Suresh	55000	20
3	Mahesh	60000	30
4	Ganesh	40000	NULL

department table

dept_id	dept_name	dept_location
10	HR	Mumbai
20	IT	Pune
30	Sales	Delhi
40	Admin	Nagpur

- Query
SELECT e.emp_name, e.salary, d.dept_name
FROM employee e
LEFT JOIN department d
ON e.dept_id = d.dept_id;
- Output

emp_name	salary	dept_name
Ramesh	45000	HR
Suresh	55000	IT
Mahesh	60000	Sales
Ganesh	40000	NULL

- Explanation
 - Joins records where employee.dept_id = department.dept_id.
 - Returns **all employees** (left table).
 - If an employee doesn't belong to any department → shows **NULL** for department details.
- Right Outer Join
 - Why we use it:
 Right Outer Join is used when we want to include all records from the _____ table (usually the secondary or reference table), even if there are no matching records in the _____ table.
 It ensures that we don't lose any data from the right table while still displaying matching rows from the left table.
 - Syntax:
SELECT alias.column_list
FROM table1 t1
INNER JOIN table2 t1
ON t1.common_column = t2.common_column;
 - Example Tables

employee table

emp_id	emp_name	salary	dept_id
1	Ramesh	45000	10
2	Suresh	55000	20
3	Mahesh	60000	30

department table

dept_id	dept_name	dept_location
10	HR	Mumbai
20	IT	Pune

- Query:
SELECT e.emp_name, e.salary, d.dept_name
FROM employee e
RIGHT JOIN department d
ON e.dept_id = d.dept_id;
- Output

emp_name	salary	dept_name
Ramesh	45000	HR
Suresh	55000	IT
Mahesh	50000	Sales
NULL	NULL	Admin

- Explanation:
 - Joins records where `employee.dept_id = department.dept_id`
 - Returns all departments (right table).
 - Employees without a matching department → NULL on left side.
- Self-Join
 - What is a Self-Join?
A Self Join is a type of join where a table is joined with itself.
It is used when the data in one table has a relationship with other rows of the same table.
In other words, the same table is treated as two separate tables temporarily — one acts as the main table, and the other as a reference table.
 - Why We Use Self Join
We use a Self-Join when:
 - A table contains hierarchical data (e.g., Employee → Manager).
 - We want to compare rows within the same table.
 - We need to find relationships among records in one table (for example, employees working under the same manager, or prerequisite courses in a course table).
 - Example Table: `employee`

emp_id	emp_name	manager_id	salary	dept_id
1	Ramesh	NULL	80000	10
2	Suresh	1	50000	10
3	Mahesh	1	48000	20
4	Dinesh	2	35000	20
5	Rajesh	2	36000	30

- Query:
SELECT e.emp_name AS Employee, m.emp_name AS Manager
FROM employee e
JOIN employee m
ON e.manager_id = m.emp_id;
- Output:

Employee	Manager
Suresh	Ramesh
Mahesh	Ramesh
Dinesh	Suresh
Rajesh	Suresh

- Explanation:
 - The employee table is joined with itself — one copy as e (Employee) and another as m (Manager).
 - The condition `e.manager_id = m.emp_id` finds each employee's manager.
 - Every manager is also an employee, which is why both are stored in the same table.
 - For example, Ramesh is a manager of Suresh and Mahesh, but he himself is also an employee in the company.
 - Employees with no manager (`manager_id = NULL`) are not shown in the result.
- Full Outer Join
 - What is a Full Outer Join?
 - A Full Outer Join combines the results of both LEFT and RIGHT joins.
 - It returns all records from both tables —
 - matching rows from both sides, and
 - unmatched rows from either side (with NULL where data is missing).
 - Why We Use Full Outer Join
 We use Full Outer Join when:
 - We want to see all data from both tables, regardless of whether it matches.
 - We don't want to lose any information while merging two datasets.

- It's useful for data reconciliation — e.g., finding which employees don't have a department assigned, or which departments have no employees yet.
- Example Table
employee

emp_id	emp_name	salary	dept_id
1	Ramesh	50000	10
2	Suresh	48000	20
3	Mahesh	47000	30
4	Dinesh	46000	NULL

department

dept_id	dept_name	dept_location
10	HR	Pune
20	IT	Mumbai
40	Sales	Delhi

- Query
MySQL doesn't support FULL OUTER JOIN directly.
You can simulate it using UNION(refer UNION topic) of LEFT JOIN and RIGHT JOIN:
SELECT e.emp_name, e.dept_id, d.dept_name, d.dept_location
FROM employee e
LEFT JOIN department d ON e.dept_id = d.dept_id

UNION

SELECT e.emp_name, e.dept_id, d.dept_name, d.dept_location
FROM employee e
RIGHT JOIN department d ON e.dept_id = d.dept_id;

- Output

emp_name	dept_id	dept_name	dept_location
Ramesh	10	HR	Pune
Suresh	20	IT	Mumbai
Mahesh	30	NULL	NULL
Dinesh	NULL	NULL	NULL
NULL	40	Sales	Delhi

- Explanation:
 - Ramesh and Suresh have matching departments — displayed normally.
 - Mahesh belongs to dept_id 30, which doesn't exist in the department table → NULL values appear for department details.

- Dinesh has no department (dept_id = NULL).
- Department Sales (dept_id 40) has no employees — so NULL values appear for employee details.
- Thus, all rows from both tables are shown — matched and unmatched.
- Summary of SQL Joins

Join Type	Description	Returns / Output	Example Use Case
CROSS JOIN	Combines every row from the first table with every row from the second table (Cartesian Product).	All possible combinations of rows from both tables.	When exploring what-if scenarios or testing combinations.
INNER JOIN	Returns only rows that have matching values in both tables.	Matching records from both tables.	Show employees working in existing departments. (Most commonly used join)
LEFT OUTER JOIN	Returns all rows from the left table and only matching rows from the right table.	All left table rows; NULLs for non-matching right rows.	Find employees without an assigned department.
RIGHT OUTER JOIN	Returns all rows from the right table and only matching rows from the left table.	All right table rows; NULLs for non-matching left rows.	Find departments with no employees.
FULL OUTER JOIN	Combines results of LEFT and RIGHT joins — returns all records from both tables.	All rows from both tables with NULLs where no match.	Display all employees and all departments, even if they don't match.
SELF JOIN	Joins a table to itself using aliases.	Matches records within the same table.	Show employees and their managers (both in same table).

- Quick Tips:
 - INNER JOIN → Most frequently used join in practice.
 - OUTER JOINS → Useful for finding missing or unmatched data.
 - SELF JOIN → Used for hierarchical relationships within a single table.
 - FULL OUTER JOIN → Helpful for data reconciliation (**though simulated in MySQL**)
- UNION and UNION ALL
 - What Are UNION and UNION ALL?
Both UNION and UNION ALL are used to combine the results of two or more SELECT statements into a single result set.
However, they differ in how they handle duplicate rows.
 - Why We Use Them
 - When data comes from multiple tables or queries having same structure (same number and type of columns).
 - To merge related data — for example, employees from two different branches or two different time periods.
 - Useful when combining reports or datasets.
 - Rules:
 - Each SELECT statement must have the same number of columns.

- The data types of corresponding columns must be compatible.
- The column order must match.

○ Example Tables

employee_mumbai

emp_id	emp_name	salary
1	Ramesh	50000
2	Suresh	48000

employee_pune

emp_id	emp_name	salary
3	Mahesh	47000
4	Suresh	48000

○ Example 1 — Using UNION

**SELECT emp_name, salary FROM employee_mumbai
UNION**

SELECT emp_name, salary FROM employee_pune;

○ Output:

emp_name	salary
Ramesh	50000
Suresh	48000
Mahesh	47000

Explanation:

UNION removes duplicate rows (Suresh, 48000 appears once).

○ Example 2 — Using UNION ALL

**SELECT emp_name, salary FROM employee_mumbai
UNION ALL**

SELECT emp_name, salary FROM employee_pune;

○ Output:

emp_name	salary
Ramesh	50000
Suresh	48000
Mahesh	47000
Suresh	48000

Explanation:

UNION ALL keeps all records, including duplicates.

- Difference Between UNION and UNION ALL

Feature	UNION	UNION ALL
Removes Duplicates	Yes	No
Performance	Slower (extra step to remove duplicates)	Faster
Result Set	Unique records only	All records including duplicates
Use Case	When duplicate rows are not needed	When you want complete data as-is

ACID Properties

So far, we've learned how to:

- Create and modify database structures using DDL
- Insert, update, and delete data using DML
- Retrieve and filter records using DQL with clauses like WHERE, ORDER BY, and GROUP BY
- Use subqueries to fetch related results from other tables
- Combine tables using JOINS
- Merge outputs from multiple queries using UNION and UNION ALL

All these commands help us work with data, but in real-world applications, these operations often form part of a larger logical task called a transaction.

- Example

When a bank transfers money from one account to another:

- ₹2000 is deducted from Account A
- ₹2000 is added to Account B

Both operations must happen together.

If one succeeds and the other fails, the database will have inconsistent data.

- That's why we need transactions that ensure all operations either complete successfully or not at all and this reliability is defined by **ACID properties**.

- ACID Properties in DBMS

A transaction is a logical unit of work consisting of one or more SQL statements.

Every transaction must satisfy the following **ACID** properties:

- A — Atomicity
 - Definition:
All steps of a transaction are treated as a single unit either all succeed or none do.
 - Example:
If ₹2000 is deducted from Account A but not added to Account B (due to a crash),
the system will undo the deduction.
All or none happens.
- C — Consistency
 - Definition:



A transaction must transform the database from one valid state to another.

All rules, constraints, and relationships must remain intact.

- Example:

Before and after the transfer, the **total money** ($A + B$) must be the same —

no data corruption or loss.

Data must remain valid.

- I — Isolation

- Definition:

Multiple transactions running at the same time must not interfere with each other.

- Example:

If two transfers occur simultaneously, each transaction should behave as if it's running alone, without seeing the intermediate results of the other.

- D — Durability

- Definition:

Once a transaction is **committed**, its changes are **permanent**, even if the system fails.

- Example:

After committing the transfer, data remains saved even if the server restarts.

- Summary

Property	Meaning	Simple Example
A — Atomicity	All or none operations execute	Deduct and deposit must both happen
C — Consistency	Data must remain valid	Total money remains same
I — Isolation	Transactions run independently	Parallel transfers don't affect each other
D — Durability	Once committed → permanent	Committed changes survive crash

We now understand the theory of transactions using ACID properties.

Let's see how we can implement and manage transactions in SQL using **TCL commands**

such as COMMIT, ROLLBACK, and SAVEPOINT.

TCL (Transaction Control Language)

TCL commands are used to **manage transactions** in a database they help us save, undo, or control groups of changes.

- COMMIT

Used to **save** all the changes made by a transaction _____ in the database.

Once committed, you _____ **roll back** the changes.

- Example:

COMMIT;

Saves all updates permanently.

- ROLLBACK

Used to _____ all the changes made in the current transaction (till the last COMMIT or SAVEPOINT).

- Example:
ROLLBACK;
Cancels uncommitted changes.
- **SAVEPOINT**
Used to **set a checkpoint** within a transaction.
You can roll back to this point **without undoing the entire transaction**.
 - Example:
SAVEPOINT step1;
ROLLBACK TO step1;
Undo only partial changes.
- **SET AUTOCOMMIT**
Determines whether each SQL statement is automatically committed after execution.

Mode	Description	Example	Analogy
SET AUTOCOMMIT = 1;	Enables autocommit — every change is saved automatically.	sql SET AUTOCOMMIT = 1;	Writing in pen — permanent instantly.
SET AUTOCOMMIT = 0;	Disables autocommit — user must explicitly use COMMIT.	sql SET AUTOCOMMIT = 0;	Writing in pencil — can erase before finalizing.

- Example workflow:
SET AUTOCOMMIT = 0;

INSERT INTO account VALUES (201, 'Riya', 5000);
UPDATE account SET balance = balance - 1000 WHERE acc_id = 201;
COMMIT; -- to finalize
ROLLBACK; -- if mistake
 - Key Point:
Autocommit OFF gives you control over when data is permanently stored.
- Quick Recap

Command	Purpose	Analogy
COMMIT	Save changes permanently	Save
ROLLBACK	Undo uncommitted changes	Undo
SAVEPOINT	Create a temporary checkpoint	Bookmark
SET AUTOCOMMIT	Auto or manual control over commits	Pencil vs Pen

- Example: Money Transfer Using TCL Commands
Imagine a banking system with a table:
Account

acc_no	holder_name	balance
101	Ramesh	20000
102	Suresh	15000

- Step 1 — Start Transaction
Ramesh wants to transfer ₹5,000 to Suresh.
START TRANSACTION;
Transaction begins — nothing is permanent yet.
- Step 2 — Deduct Amount from Ramesh
UPDATE account
SET balance = balance - 5000
WHERE acc_no = 101;
Ramesh's balance temporarily becomes ₹15,000.
(Change is not permanent yet.)
- Step 3 — Create a SAVEPOINT
SAVEPOINT after_deduction;
Savepoint created — can roll back to this point if needed.
- Step 4 — Add Amount to Suresh
UPDATE account
SET balance = balance + 5000
WHERE acc_no = 102;
Suresh's balance temporarily becomes ₹20,000.
- Step 5 — Something Goes Wrong
If the second update fails (like network or constraint issue):
ROLLBACK TO after_deduction;
Only Ramesh's deduction is undone.
Balances return to original values.
- Step 6 — Commit Successfully
COMMIT;
Both updates are permanently saved.
Transaction ends successfully.
- How It Follows ACID Properties

Property	Meaning	Relation in Example
Atomicity	All or none	Either both updates happen, or none
Consistency	Valid state maintained	Total money remains ₹35,000
Isolation	Transaction independent	Others can't see partial updates
Durability	Permanent after commit	Data stays safe even after system restart

DCL (Data Control Language)

- So far, we've learned how **transactions** are handled safely using TCL commands and how **ACID properties** ensure data integrity during changes.
- But in a real-world database, not every user should have the power to **modify, delete, or access** all data.
That's where **Data Control Language (DCL)** comes in — it focuses on **security and access control**.
- With DCL, we can decide:
 - Who can access the data, and
 - What actions they are allowed to perform.
- Main DCL Commands:
 - _____ — Creates a new database user.

- _____ – Gives permissions (privileges) to a user.
- _____ – Removes permissions from a user.
- **CREATE USER Command**
Before assigning permissions, we must first **create a user account** in MySQL.
 - **Syntax:**
CREATE USER 'username'@'hostname' IDENTIFIED BY 'password';
 - Examples:
CREATE USER 'Ramesh'@'localhost' IDENTIFIED BY 'ramesh123';
CREATE USER 'Priya'@'%' IDENTIFIED BY 'priya@123';
'localhost' means the user can log in **only from the same machine**.
'%' allows the user to connect **from any host** (remote access).
- **GRANT Command**
Used to **give specific privileges** to a user.
 - Syntax:
GRANT privilege_name ON object_name TO 'username'@'hostname';
 - Examples:
GRANT SELECT, INSERT ON employee TO 'Ramesh'@'localhost';
Ramesh can _____ **and** _____ data into the employee table.
- **Granting All Privileges**
You can give **full access** using **ALL PRIVILEGES**.
 - Example 1 – On a Table:
GRANT ALL PRIVILEGES ON employee TO 'Ramesh'@'localhost';
 - Example 2 – On a Database:
GRANT ALL PRIVILEGES ON company_db.* TO 'Ramesh'@'localhost';
 - Example 2 – On a Database:
GRANT ALL PRIVILEGES ON company_db.* TO 'Ramesh'@'localhost';
 - Example 3 – Allow User to Grant to Others:
GRANT ALL PRIVILEGES ON company_db.* TO 'Ramesh'@'localhost' WITH GRANT OPTION;
Now Ramesh can also **grant privileges** to other users.
- **Common Privileges in MySQL**

Privilege	Applies To	Description / What it Allows	Example Command
SELECT	Table / View	Allows user to read data from a table	GRANT SELECT ON employee TO 'Amit'@'localhost';
INSERT	Table	Allows user to insert new records	GRANT INSERT ON employee TO 'Priya'@'localhost';
UPDATE	Table	Allows user to modify existing records	GRANT UPDATE ON employee TO 'Ramesh'@'localhost';
DELETE	Table	Allows user to delete records	GRANT DELETE ON employee TO 'Ramesh'@'localhost';
CREATE	Database / Table	Allows user to create new tables or databases	GRANT CREATE ON company_db.* TO 'Priya'@'localhost';

Privilege	Applies To	Description / What it Allows	Example Command
ALTER	Table	Allows changing structure of existing tables	GRANT ALTER ON employee TO 'Ramesh'@'localhost';
DROP	Database / Table	Allows user to delete tables or databases	GRANT DROP ON company_db.* TO 'Amit'@'localhost';
INDEX	Table	Allows creating and removing indexes	GRANT INDEX ON employee TO 'Ramesh'@'localhost';
ALL PRIVILEGES	Database / Table	Grants all available privileges	GRANT ALL PRIVILEGES ON company_db.* TO 'Ramesh'@'localhost';
WITH GRANT OPTION	—	Allows user to grant privileges to others	GRANT ALL PRIVILEGES ON company_db.* TO 'Ramesh'@'localhost' WITH GRANT OPTION;

- **REVOKE Command**

Used to **remove privileges** given earlier.

Note:

Even if a user has been granted **ALL PRIVILEGES** on a table, you can still **revoke specific privileges** from that user later.

- Syntax:

REVOKE privilege_name ON object_name FROM 'username'@'hostname';

- Example 1:

REVOKE INSERT ON employee FROM 'Ramesh'@'localhost';

Removes Ramesh's permission to insert data.

- Example 2:

REVOKE ALL PRIVILEGES ON company_db.* FROM 'Ramesh'@'localhost';

Removes all rights from Ramesh in the company_db database.

VIEW

- **What is VIEW**

A **VIEW** is a **virtual table** that shows data from one or more tables.

It does **not store data physically**, it only stores the **query** used to fetch the data.

You can think of a view as a *window* through which you can see only selected columns and rows of a table.

- **Why Use a View?**

- **Security:** Hide sensitive columns (e.g., salary).
- **Simplicity:** Simplify complex queries by saving them as a view.
- **Consistency:** Ensure users always see up-to-date data.
- **Data Independence:** Underlying table changes don't affect the view name or structure much.

- Syntax:

**CREATE VIEW view_name AS
SELECT column1, column2, ...**

FROM table_name
WHERE condition;

- Example:
Suppose we have an `employee` table:

emp_id	emp_name	dept_id	salary	location
101	Ramesh	10	45000	Mumbai
102	Suresh	20	55000	Pune
103	Mahesh	10	60000	Mumbai

- Create a view showing employees from Mumbai:
Mumbai_Employees AS

SELECT emp_id, emp_name, salary
FROM employee
WHERE location = 'Mumbai';

- Now display data using the view:
SELECT * FROM Mumbai_Employees;
- Output:

emp_id	emp_name	salary
101	Ramesh	45000
103	Mahesh	60000

- Modify a View:
CREATE OR REPLACE VIEW Mumbai_Employees AS
SELECT emp_id, emp_name, salary, dept_id
FROM employee
WHERE location = 'Mumbai';
- Drop a View:
DROP VIEW Mumbai_Employees;

A **View** is like a *filtered lens* over a table — you decide *what part of the data others can see*, without giving them access to the entire table.

- Types of Views

Type	Description	Example
Simple View	Created from a single table, without using functions or group operations.	CREATE VIEW Emp_View AS SELECT emp_id, emp_name FROM employee;
Complex View	Created from multiple tables or includes functions, joins, or group by.	CREATE VIEW Dept_Salary AS SELECT d.dept_name, AVG(e.salary) FROM employee e JOIN department d ON e.dept_id = d.dept_id GROUP BY d.dept_name;

- Updatable(Simple) and Non-Updatable(Complex) Views

So far, we've learned how to create and use a view, a virtual table that displays data derived from one or more base tables.

However, when we start performing **DML operations** (like INSERT, UPDATE, or DELETE) on a view, an important question arises:

Can we modify the data in a view just like we do in a table?

The answer depends on **how the view is created**.

Some views allow DML operations — these are called **updatable views**(Simple), while others do not — known as **non-updatable views**(Complex).

- Updatable Views

- You can perform **INSERT**, **UPDATE**, and **DELETE** operations on an updatable view.
- Any change made through the view is reflected in the base table.
- A view is updatable only if it is created using a **single base table** and does **not include**:
 - Aggregate functions (SUM, AVG, etc.)
 - DISTINCT
 - GROUP BY or HAVING
 - UNION or JOIN
 - Subqueries

- Example:

```
CREATE VIEW emp_view AS
SELECT id, emp_name, salary, dept_id
FROM employee;
```

- Now we can update or delete through the view:

```
UPDATE emp_view SET salary = 60000 WHERE id = 101;
```

This will update the salary in both emp_view and the base table employee.

- Non-Updatable Views

- You cannot perform DML operations on non-updatable views.
- These views are mainly used for **displaying or reporting** data.
- A view becomes non-updatable when it contains:
 - Aggregate functions
 - DISTINCT keyword
 - GROUP BY or HAVING clauses
 - JOIN or UNION
 - Subqueries

- Example:

```
CREATE VIEW dept_avg_salary AS
SELECT dept_id, AVG(salary) AS avg_salary
FROM employee
GROUP BY dept_id;
```

- Attempting to modify through this view:

```
UPDATE dept_avg_salary SET avg_salary = 70000 WHERE dept_id = 2;
```

This will give an **error**, because aggregate results cannot be modified.

INDEX in SQL

We have seen how **views** help simplify data access by creating virtual tables.

However, when the data in our tables grows very large, **retrieving records quickly** becomes equally important.

That's where **indexes** come in, they help speed up data retrieval just like an index in a book helps you find a topic quickly.

- Definition
 - An **index** in SQL is a **database object** used to **improve the speed of data retrieval** from a table.
 - When we search for something in a large table, without an index, MySQL checks each and every row (slow).
 - With an index, MySQL can jump directly to the matching rows (fast).
 - In simple words:
Index makes searching faster — just like the index page of a book helps you quickly find a topic.

- Analogy
Think about your dictionary:
 - Without index → you flip through every page to find a word.
 - With index → you directly jump to the correct page number.

That's exactly how an **index in SQL** works!

- Syntax

_____ index_name
ON table_name (column_name);

- Example:

_____ idx_salary _____ employee(salary);

- To Remove an Index
DROP INDEX index_name ON table_name;

- Example:
DROP INDEX idx_salary ON employee;

- Types of Indexes

Index Type	Description	Example	Notes
Primary Index	Automatically created when you define a PRIMARY KEY . It uniquely identifies each record.	CREATE TABLE employee(emp_id INT PRIMARY KEY, emp_name VARCHAR(50));	Only one primary index per table. Values must be unique and NOT NULL.
Unique Index	Prevents duplicate values in a column.	CREATE UNIQUE INDEX idx_email ON employee(email);	Allows only one row per unique value.
Simple Index	Created on a single column to speed up queries.	CREATE INDEX idx_name ON employee(emp_name);	Improves performance for WHERE clauses using that column.
Composite Index	Index on multiple columns together.	CREATE INDEX idx_name_dept ON employee(emp_name, dept_id);	Useful when queries use both columns in WHERE conditions.



- Important Points
 - Index makes **SELECT** queries _____.
 - But it makes _____, _____, and _____ slightly slower, because the index also needs to be updated.
 - When you create a **PRIMARY KEY**, MySQL automatically creates a **unique index**.
 - Avoid creating too many indexes, it may reduce performance.

FirstBitSolutions.com
... Learn IT, Bit by Bit ...

PL/SQL (Procedural SQL in MySQL)

So far, we've learned how SQL helps us create, manipulate, and query data using statements like SELECT, INSERT, UPDATE, and DELETE.

However, SQL on its own executes one statement at a time — it doesn't allow loops, conditions, or reusable logic.

That's where PL/SQL (Procedural Language extensions to SQL) comes in.

- What is PL/SQL?
 - PL/SQL (in MySQL often called procedural SQL) adds programming features like:
 - Variables
 - Conditions (IF, CASE)
 - Loops (WHILE, LOOP, REPEAT)
 - Procedures and Functions
 - Error handling (DECLARE ... HANDLER)
 - It allows you to group SQL statements together and control how they execute — just like writing a mini program inside the database.
- Analogy:

Think of SQL as giving individual instructions, while PL/SQL lets you write a complete script or program to perform a task step by step.

Stored Procedure in MySQL

- What is a Stored Procedure?

A **stored procedure** is a block of SQL statements that is saved in the database and can be executed whenever needed.

It helps in reusing code and performing tasks in a single call.

 - Simple idea:

Instead of typing multiple SQL commands again and again, you can store them in a *procedure* and just call it when needed.
 - Syntax

_____ #

CREATE PROCEDURE procedure_name()

BEGIN

 -- SQL statements

_____ #

DELIMITER ;

- Why Change the DELIMITER?

By default, MySQL ends every command with a semicolon (;).

But inside a procedure, we write multiple statements ending with ;.

So, we temporarily change the delimiter to something else (like #) to tell MySQL:

“Don't end here — the procedure will end only when you see #.”

After creating the procedure, we switch back to the normal ;.

- Syntax:

DELIMITER #

CREATE PROCEDURE procedure_name()

```
BEGIN
  -- SQL statements
END #
```

DELIMITER ;

- Creating a Simple Procedure:

DELIMITER #

_____ **displayEmployees()**

BEGIN

```
SELECT * FROM employee;
```

END #

DELIMITER ;

To execute:

CALL displayEmployees();

Output: Displays all records from the employee table.

- Managing Stored Procedures

Once a procedure is created, we may need to **view**, **delete**, or **safely recreate** it.

Here are some useful commands and best practices:

- View Existing Procedures

To list all procedures in the current database:

SHOW PROCEDURE STATUS WHERE Db = 'your_database_name';

you can use another way also to list the current databases:

SHOW PROCEDURE STATUS WHERE Db = DATABASE();

- To see the code of a specific procedure:

SHOW CREATE PROCEDURE procedure name;

- Delete (Drop) a Procedure

To remove a procedure permanently:

DROP PROCEDURE procedure_name;

- Safe Deletion Using IF EXISTS

If you try to drop a procedure that doesn't exist, MySQL throws an error.

To avoid that, use the IF EXISTS clause.

Syntax:

DROP PROCEDURE IF EXISTS procedure name;

Example:

DROP PROCEDURE IF EXISTS get_student_details;

This ensures that your script runs safely even if the procedure is missing.

- Summary

Task	Command
View all procedures	SHOW PROCEDURE STATUS
View a specific procedure	SHOW CREATE PROCEDURE
Delete procedure	DROP PROCEDURE
Safe delete	DROP PROCEDURE IF EXISTS

- Stored Procedure Parameters

A stored procedure can take parameters to make it dynamic and reusable.

There are **three types of parameters**:

Parameter Type	Direction	Description
IN	Input	Used to pass a value into the procedure.
OUT	Output	Used to return a value from the procedure.
INOUT	Input + Output	Used to send a value , modify it, and return the modified value .

- IN Parameter

Used to send a value *to* the procedure (read-only inside).

DELIMITER #

```
CREATE PROCEDURE getEmployeeByDept(IN deptid INT)
BEGIN
```

```
    SELECT emp_name, salary, dept_id
    FROM employee
    WHERE dept_id = deptid;
```

```
END #
```

DELIMITER ;

Execute:

```
CALL getEmployeeByDept(10);
```

Output: All employees from department 10.

- OUT Parameter

Used when you want the procedure to **return a value**.

DELIMITER #

```
CREATE PROCEDURE getTotalEmployees(OUT total INT)
BEGIN
```

```
    SELECT COUNT(*) INTO total
    FROM employee;
```

```
END #
```

DELIMITER ;

Execute:

```
CALL getTotalEmployees(@t);
```

```
SELECT @t AS 'Total Employees';
```

Output:

Total Employees
8

- INOUT Parameter

Used to send a value, modify it, and return the updated value.

DELIMITER #

```
CREATE PROCEDURE increaseSalary(INOUT sal INT)
BEGIN
    SET sal = sal + 5000;
END #
```

DELIMITER ;

Execute:

```
SET @mysal = 40000;
```

```
CALL increaseSalary(@mysal);
```

```
SELECT @mysal AS 'Updated Salary';
```

Output:

Updated Salary
45000

- Summary of Stored Procedures

Feature	Description
Definition	Block of SQL statements stored in the database
Execution	Run using <code>CALL procedure_name()</code>
Parameters	IN, OUT, INOUT
Benefits	Reusability, better performance, security, consistency

Control Flow Statements in MySQL

In the previous section, we learned how to **create stored procedures** — reusable blocks of SQL code that help us organize logic and reduce repetition.

However, many real-world tasks require **decision-making** (for example, checking a condition) or **repetition** (executing something multiple times).

To make procedures more *dynamic* and *intelligent*, MySQL provides **Control Flow Statements**, similar to what we use in programming languages like C, Python or Java.

Using these statements, we can:

- Execute a part of code **only when a condition is true**
- Repeat a task **multiple times** until a condition is met
- **Control** the flow of logic within stored programs

Let's now explore these control flow statements.

- **IF – ELSEIF – ELSE**

Used for decision-making based on conditions.

- Syntax:

```
IF condition THEN
  statement;
```

```
ELSEIF condition THEN
  statement;
```

```
ELSE
  statement;
```

```
END IF;
```

- Example:

```
DELIMITER #
```

```
CREATE PROCEDURE check_salary(IN sal INT)
```

```
BEGIN
```

```
  IF sal > 50000 THEN
```

```
    SELECT 'High Salary';
```

```
  ELSEIF sal BETWEEN 30000 AND 50000 THEN
```

```
    SELECT 'Average Salary';
```

```
  ELSE
```

```
    SELECT 'Low Salary';
```

```
  END IF;
```

```
END #
```

```
DELIMITER ;
```

- Execute:

```
CALL check_salary(45000);
```

```
Output: Average Salary
```

- **CASE Statement**

Used when multiple conditions must be tested — like a switch statement in other languages.

- Syntax:

```
CASE
```

```
  WHEN condition1 THEN result1;
```

```
  WHEN condition2 THEN result2;
```

```
  ELSE resultN;
```

```
END CASE;
```

- Example:

```
DELIMITER #
```

```
CREATE PROCEDURE grade(IN marks INT)
```



```
BEGIN
CASE
  WHEN marks >= 90 THEN SELECT 'Grade A';
  WHEN marks >= 75 THEN SELECT 'Grade B';
  WHEN marks >= 60 THEN SELECT 'Grade C';
  ELSE SELECT 'Fail';
END CASE;
```

```
END #
DELIMITER ;
```

- Execute:
CALL grade(82);
Output: Grade B

- **LOOP**

Used to execute a block of code repeatedly.

Must use **LEAVE** statement to exit the loop manually.

- Example:
DELIMITER #
CREATE PROCEDURE print_numbers()
BEGIN
 DECLARE x INT DEFAULT 1;
 num_loop: LOOP
 SELECT x;
 SET x = x + 1;
 IF x > 5 **THEN**
 LEAVE num_loop;
 END IF;
 END LOOP num_loop;
END #
DELIMITER ;

- Execute:
CALL print_numbers();
- Output:
1
2
3
4
5

- **WHILE Loop**

Executes a block of statements **while** the condition is true.

- Example:
DELIMITER #
CREATE PROCEDURE while_example()
BEGIN
 DECLARE i INT DEFAULT 1;
 WHILE i <= 5 **DO**
 SELECT CONCAT('Iteration: ', i);
 SET i = i + 1;
 END WHILE;
END #
DELIMITER ;

- Execute:
CALL while_example();
- **Output:**
Iteration: 1
Iteration: 2
Iteration: 3
Iteration: 4
Iteration: 5
- **REPEAT Loop**
Similar to WHILE, but it executes the code **first**, then checks the condition.
 - Example:
DELIMITER #
CREATE PROCEDURE repeat_example()
BEGIN
 DECLARE i INT DEFAULT 1;
 REPEAT
 SELECT CONCAT('Value: ', i);
 SET i = i + 1;
 UNTIL i > 5
 END REPEAT;
END #
DELIMITER ;
 - Execute:
CALL repeat_example();
 - Output:
Value: 1
Value: 2
Value: 3
Value: 4
Value: 5
- **Summary:**

Statement	Purpose	Key Feature
IF	Checks a condition	Simple decision making
CASE	Tests multiple conditions	Acts like switch-case
LOOP	Executes repeatedly until exited	Needs LEAVE to stop
WHILE	Runs while condition is true	Checks condition first
REPEAT	Runs until condition is true	Executes first, checks later

Stored Function in MySQL

Earlier, we learned stored procedures, which are used to perform a set of operations sometimes with parameters but they don't necessarily return a value.

Now imagine a situation where:

- You don't want to just perform an operation,
- But you want to calculate something and return a result (like total salary, bonus, or tax).

That's where stored functions come in.

- **Definition:**

A stored function in MySQL is a stored program that accepts parameters, performs calculations or operations, and returns a single value.

Stored Functions are mainly used in **SELECT**, **WHERE**, or other SQL statements.

- **Syntax:**

DELIMITER #

```
_____ function_name(parameter_list)
_____ datatype
```

BEGIN

-- declare variables

-- perform operations

RETURN value;

END #

DELIMITER ;

- **Example: Calculate Bonus**

Let's create a stored function that returns 10% bonus of an employee's salary.

DELIMITER #

CREATE FUNCTION calc_bonus(salary DECIMAL(10,2))

RETURNS DECIMAL(10,2)

DETERMINISTIC

BEGIN

_____ bonus DECIMAL(10,2);

SET bonus = salary * 0.10;

_____ bonus;

END #

DELIMITER ;

- **Execute / Use stored function**

You can call a stored function like this:

SELECT calc_bonus(50000);

- **Output:**

5000.00

- **Or use it with a table:**

SELECT emp_name, salary, calc_bonus(salary) **AS** bonus

FROM employee;

- **Managing functions**

Just like procedures, functions can also be **viewed**, **deleted**, or **safely recreated** using similar commands.

Let's explore how to manage them in MySQL.

- View Existing Functions

To see all functions in the current database:

SHOW FUNCTION STATUS WHERE Db = 'your_database_name';

you can use another way also to list the current databases:

SHOW FUNCTION STATUS WHERE Db = DATABASE();

- To view the code (definition) of a particular function:

SHOW CREATE FUNCTION function_name;

- Delete (Drop) a Function

To remove a function permanently:

DROP FUNCTION function_name;

- Safe Deletion Using IF EXISTS

To avoid an error when the function doesn't exist, use the IF EXISTS clause.

Syntax:

DROP FUNCTION IF EXISTS function_name;

Example:

DROP FUNCTION IF EXISTS calculate_bonus;

This ensures your script won't fail even if the function is already deleted.

- Summary

Task	Command
View all functions	SHOW FUNCTION STATUS
View a specific function	SHOW CREATE FUNCTION
Delete function	DROP FUNCTION
Safe delete	DROP FUNCTION IF EXISTS

- Difference Between Procedure and Function

Feature	Procedure	Function
Returns a value	Optional	Mandatory (must return one value)
Called using	CALL procedure_name();	SELECT function_name();
Used in SQL statements	No	Yes (SELECT, WHERE, etc.)
Parameters allowed	IN, OUT, INOUT	Only IN
Purpose	Perform an action	Compute and return a value
Example use	Insert, update, delete operations	Calculations, data transformations

Triggers in MySQL

So far:

- **Procedures:** perform a set of actions when *you* call them.
- **Functions:** perform calculations and *return values*.

But what if you want MySQL to automatically perform an action whenever something happens to a table — for example, when a record is inserted, updated, or deleted?

That's where Triggers come in.

- **Definition:**

A **trigger** is a stored program that is **automatically executed** (fired) by MySQL when a specific event occurs on a table, such as:

- _____
- _____
- _____

You don't call a trigger manually, MySQL executes it *automatically* when the event happens.

- **Syntax of a Trigger**

DELIMITER #

```
CREATE TRIGGER trigger_name
BEFORE | AFTER INSERT | UPDATE | DELETE
ON table_name
FOR EACH ROW
BEGIN
    -- actions to perform
END #
```

DELIMITER ;

- **Example 1: Log Insert Operations**

Let's say you have an `employee` table, and you want to automatically record any new employee added into a log table called `employee_audit`.

Step 1: Create tables

```
CREATE TABLE employee (
    emp_id INT PRIMARY KEY AUTO_INCREMENT,
    emp_name VARCHAR(50),
    salary DECIMAL(10,2)
);
```

```
CREATE TABLE employee_audit (
    audit_id INT PRIMARY KEY AUTO_INCREMENT,
    emp_name VARCHAR(50),
    action_time DATETIME
);
```

Step 2: Create trigger

DELIMITER #

```
_____ trg_after_insert_employee
_____ employee
FOR EACH ROW
BEGIN
```

```
INSERT INTO employee_audit(emp_name, action_time)
VALUES (NEW.emp_name, NOW());
END #
```

DELIMITER ;

Now, whenever you insert a record into `employee`:

```
INSERT INTO employee(emp_name, salary) VALUES ('Amit', 50000);
```

The trigger will automatically insert a record into `employee_audit`.

- Understanding OLD and NEW

Context	Meaning
NEW.column_name	The new value being inserted or updated
OLD.column_name	The existing value before update or delete

- Example:
 - In an AFTER INSERT trigger → use NEW.column_name
 - In an AFTER DELETE trigger → use OLD.column_name
 - In an AFTER UPDATE trigger → you can use both
- Example 2: Track Salary Changes

DELIMITER

```
CREATE TRIGGER trg_after_update_salary
AFTER UPDATE ON employee
FOR EACH ROW
BEGIN
  IF OLD.salary <> NEW.salary THEN
    INSERT INTO employee_audit(emp_name, action_time)
    VALUES (CONCAT('Salary changed for ', NEW.emp_name), NOW());
  END IF;
END #
```

DELIMITER ;

- Managing Triggers

Triggers, like procedures and functions, can also be **viewed**, **deleted**, and **safely recreated** in MySQL.

This helps you maintain control over automation that happens when records are inserted, updated, or deleted.

 - View Existing Triggers

To see all triggers present in the current database:

```
SHOW TRIGGERS;
```

To see triggers of a specific table only:

```
SHOW TRIGGERS LIKE 'table_name';
```

This helps you check which tables have automatic actions (triggers) defined.
 - View Trigger Definition

To view the full definition (the SQL code inside a trigger):

```
SHOW CREATE TRIGGER trigger_name;
```
 - Delete (Drop) a Trigger

To permanently remove a trigger:

```
DROP TRIGGER trigger_name;
```

- Safe Deletion Using IF EXISTS
To avoid an error if the trigger doesn't exist, use:
DROP TRIGGER IF EXISTS trigger_name;
Example:
DROP TRIGGER IF EXISTS before_salary_update;
This prevents your script from failing during re-execution or migration.
DELIMITER #

```
CREATE TRIGGER before_salary_update
BEFORE UPDATE ON employee
FOR EACH ROW
BEGIN
    INSERT INTO audit_log(action, old_salary, new_salary)
    VALUES('UPDATE', OLD.salary, NEW.salary);
END #
```

DELIMITER ;

This pattern ensures your trigger always recreates cleanly.

- Summary

Task	Command
View all triggers	SHOW TRIGGERS;
View trigger definition	SHOW CREATE TRIGGER trigger_name;
Delete trigger	DROP TRIGGER trigger_name;
Safe delete	DROP TRIGGER IF EXISTS trigger_name;

- Types of Triggers

Timing	Event	Description
BEFORE INSERT	Runs before inserting a row	Used for validation
AFTER INSERT	Runs after inserting a row	Used for logging
BEFORE UPDATE	Runs before updating a row	Used for modification checks
AFTER UPDATE	Runs after updating a row	Used for audit/logging
BEFORE DELETE	Runs before deleting a row	Used for prevention
AFTER DELETE	Runs after deleting a row	Used for maintaining history

- Notes on Triggers

Do's (Best Practices)

- Use triggers for automation
Perfect for logging, auditing, maintaining history, or enforcing simple business rules.
- Use meaningful names
Example: `trg_after_insert_employee` or `trg_before_delete_order`
Naming clearly indicates what the trigger does.
- Keep the logic simple and short
Complex operations inside triggers can slow down queries.

- Add comments in trigger code
Helps others (and you later) understand why the trigger exists.
- Test carefully before production
Triggers run automatically — once created, they fire every time the event occurs.
- Use `OLD` and `NEW` properly
`NEW` for inserted/updated values
`OLD` for existing/deleted values
- Document all triggers
Keep a list of all triggers per table; otherwise, debugging becomes hard.
- Don'ts (Common Mistakes)
 - Don't use triggers for business logic
Use *stored procedures* or *application code* for main business rules.
Triggers are for *automated reactions*, not for controlling main program flow.
 - Don't modify the same table that fired the trigger
Can lead to *recursive loops* or *mutating table errors*.
 - Don't perform long-running queries inside triggers
Triggers run automatically — slow logic slows down your inserts/updates too.
 - Don't depend on trigger execution order
MySQL doesn't guarantee which trigger (`BEFORE/AFTER`) runs first if multiple exist for same event.
 - Don't ignore error handling
Use conditions or control statements to avoid inserting wrong data inside triggers.
 - **Don't use triggers for cross-table validation** unless necessary
Keep triggers table-focused to avoid unwanted dependencies.
 - Don't assume users will “remember” there's a trigger
Always mention it in documentation or comments, else future updates might behave unexpectedly.

Cursor in MySQL

- What is a Cursor?
A cursor is a temporary work area in memory created by MySQL to store the result of a query and allow row-by-row processing of that data.
Cursors are mainly used inside stored procedures or functions when you need to perform operations on each row _____.
- Why We Need a Cursor
In SQL, most operations (like `UPDATE`, `DELETE`, `INSERT`, `SELECT`) are set-based they affect all matching rows at once.
However, sometimes we need to process rows one by one, especially when:
 - Each row needs custom logic or calculations.
 - You need to log or call another procedure for each record.
 - You must perform multiple SQL statements per row.
 In such situations, a cursor is used to fetch each record sequentially.
- Use Case
 - Student Result Processing System
Problem Statement (very important)
We have a table of student marks.
Rules:

For each student:

1. Calculate grade based on marks
2. Insert a result message into result_log
3. If student fails, also insert record into retest_list

Notice:

- One row → multiple actions
- Conditional insert into different tables
- Cannot be expressed in single UPDATE or INSERT

This is where cursor is genuinely needed

Tables

Student

id | name | marks

result_log

student_id | message

retest_list

student_id | reason

Why CASE / set-based SQL is NOT enough here

You can:

- Calculate grade using CASE not enough
- Insert into two different tables conditionally not possible in one query

This is procedural logic, not just data transformation.

Example:

DELIMITER #

CREATE PROCEDURE process_results()

BEGIN

DECLARE done INT DEFAULT 0;

DECLARE s_id INT;

DECLARE s_name VARCHAR(50);

DECLARE s_marks INT;

DECLARE grade CHAR(1);

DECLARE cur CURSOR FOR

```

SELECT id, name, marks FROM student;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET done =
1;

OPEN cur;

read_loop: LOOP
    FETCH cur INTO s_id, s_name, s_marks;

    IF done = 1 THEN
        LEAVE read_loop;
    END IF;

    -- Decide grade
    IF s_marks >= 60 THEN
        SET grade = 'A';
    ELSEIF s_marks >= 40 THEN
        SET grade = 'B';
    ELSE
        SET grade = 'F';
    END IF;

    -- Insert result log
    INSERT INTO result_log
    VALUES (s_id, CONCAT(s_name, ' got grade ', grade));

    -- Extra action ONLY for failed students
    IF grade = 'F' THEN
        INSERT INTO retest_list
        VALUES (s_id, 'Failed – needs retest');
    END IF;

END LOOP;
CLOSE cur;
END#

```

- Summary

Concept	Description
Cursor	Used to process rows one by one
OPEN	Initializes and prepares the cursor
FETCH	Retrieves next row into variables
CLOSE	Releases the cursor
HANDLER	Detects when cursor reaches end
Use When	You need per-row logic (logging, multiple operations, etc.)



FirstBitSolutions.com
... Learn IT, Bit by Bit ...